Information and Software Technology 189 (2026) 107915

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Review article

Artificial intelligence for source code understanding tasks: A systematic mapping study

Dzikri Rahadian Fudholi *, Andrea Capiluppi

Bernoulli Institute for Mathematics, Computer Science and Artificial Intelligence, University of Groningen, Nijenborgh 9, Groningen, 9747 AG, Netherlands

ARTICLE INFO

Dataset link: https://zenodo.org/records/16947597

Keywords:
Natural language processing
Embedding
Text preprocessing
Machine learning
Program semantics

ABSTRACT

Context: Artificial intelligence (AI) techniques, particularly natural language processing (NLP) and machine learning (ML), are increasingly used to support source code understanding, an essential activity in software engineering.
Objective: This systematic mapping study investigates how these techniques are applied, guided by four Research Questions (RQs) focusing on the types of tasks, embedding methods & preprocessing techniques used, machine learning models employed, and existing research gaps.
Methods: A review of 227 peer-reviewed studies identifies trends and provides a structured mapping addressing each RQ.
Results: The findings reveal a dominant shift toward deep learning, especially transformer-based and graph-based models, highlighting underexplored areas such as explainability.
Conclusion: This study provides a task-based classification and offers insights and directions for future research in AI-enabled source code understanding, supporting both researchers and practitioners.

Contents

1. Introduction 2
2. Related work 3
   2.0.1. Detection tasks 3
   2.0.2. Searching and understanding tasks 3
   2.0.3. Generation and summarization tasks 3
   2.0.4. Foundational studies 3
   2.0.5. Positioning of this study 3
3. Methodology 3
   3.1. Research questions 4
   3.2. Relevant study identification 4
      3.2.1. Search strategy 4
      3.2.2. Cleaning and initial screening 4
      3.2.3. Snowballing process 4
   3.3. Study quality assessment 4
   3.4. Data extraction 5
   3.5. Data synthesis 6
4. Results 6
   4.1. General trend 6
   4.2. Task categorization (RQ1) 6
      4.2.1. Detection subtasks 7
      4.2.2. Search & understanding subtasks 7
      4.2.3. Generation & summarization subtasks 7
   4.3. Preprocessing and embedding (RQ2) 8
      4.3.1. Preprocessing 8

* Corresponding author.
E-mail address: dzikri.r.f@ugm.ac.id (D.R. Fudholi).

https://doi.org/10.1016/j.infsof.2025.107915
Received 11 June 2025; Received in revised form 26 August 2025; Accepted 7 October 2025
Available online 10 October 2025
0950-5849/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

4.3.2. Embedding ... 9
4.4. Modeling (RQ3) ... 10
4.4.1. Modeling for detection ... 10
4.4.2. Modeling for search & understanding ... 11
4.4.3. Modeling for generation & summarization ... 11
5. Discussion ... 11
5.1. RQ1: Task categorization ... 11
5.1.1. Detection ... 11
5.1.2. Search & understanding ... 12
5.1.3. Generation and summarization ... 12
5.2. RQ2: Preprocessing and embedding ... 12
5.2.1. Preprocessing ... 13
5.2.2. Embedding ... 13
5.3. RQ3: Modeling ... 13
5.3.1. Statistical machine learning ... 13
5.3.2. Neural network ... 13
5.3.3. Performance measurement ... 14
5.4. RQ4: Gaps, challenges and opportunities ... 14
5.4.1. Challenges and opportunities for researchers ... 14
5.4.2. Challenges and opportunities for practitioners ... 15
6. Threat to validity ... 15
7. Conclusion ... 15
CRediT authorship contribution statement ... 15
Declaration of competing interest ... 16
Acknowledgments ... 16
Data availability ... 16
References ... 16

1. Introduction

Source code within the Software Engineering (SE) domain can be viewed as a specialized form of language. Although programmers write code influenced by personal preferences and styles, similarly to natural languages [1], programming languages are more structured and rigid due to the necessity of accurate communication with machines [2]. Despite these structural constraints, natural language elements frequently appear in source code, particularly in comments and naming conventions, introducing subjective aspects into otherwise formal language constructs [3]. These two roles highlight the need to approach source code as a structured language for machine communication and an expressive language for human understanding, an information exchange between human programmers and computers.

Communication in source code depends on numerous factors, such as naming conventions, structural patterns, commenting practices, and the specific programming language used. These elements are shaped by individual developer preferences, team norms, and project-specific requirements, introducing high variability in how meaning is constructed and perceived. This variability leads to complexity, as even semantically similar code fragments may differ substantially in syntax across teams or projects. Consequently, interpreting and understanding source code becomes time-consuming and resource-intensive, particularly in large or collaborative environments where consistent understanding is vital for maintainability and evolution [3]. Furthermore, the challenge came from the dual nature of source code: it must be syntactically precise for machine execution, yet semantically rich for human readability [4]. These competing demands often cause inconsistencies and ambiguity in code communication, hindering effective understanding. In this context, NLP emerges as a powerful tool to bridge the gap between syntax and semantics. By analyzing naming patterns, extracting contextual information and normalizing structural variations, NLP reduces the thinking burden involved in code comprehension. Through such techniques, NLP contributes directly to the broader goal of automated and scalable source code understanding, making it possible to uncover developer intent, detect patterns and support intelligent software analysis. Although the communication embedded in source code is complex, the application of NLP offers promising pathways to make this communication more accessible, actionable and interpretable.

Applying NLP methodologies to source code presents unique challenges not commonly encountered in natural language processing. Unlike spoken or written languages, programming languages are designed for precise syntactic interpretation by machines [4]. This challenge is intensified by the diversity of programming languages, each with its own syntax and semantic rules [5]. Moreover, while source code shares some similar properties with natural language, its structural constraints and abstraction levels require specially adapted models for effective processing [6]. These distinctions confirm the need for specialized NLP techniques tailored to the characteristics of source code. A study is needed to lay the groundwork for examining how such techniques have been adopted in recent research.

To explore how Artificial Intelligence, in this case NLP, contributes to source code understanding, this paper conducts a Systematic Mapping Study (SMS) designed to systematically categorize the use of NLP techniques in software engineering tasks. This study follows established guidelines for systematic reviews in software engineering [7,8] and applies a backward and forward snowballing strategy [9] to ensure comprehensive coverage of the relevant literature. Through this SMS, the research aims to map existing studies across tasks such as detection, searching & understanding and generation & summarization. In addition to exploring task types, this study also examines the embedding methods, preprocessing techniques and machine learning models employed within each task type, offering a structured view of how these components interact. By analyzing these studies, the paper identifies methodological patterns, highlights underexplored areas and points to opportunities for future work.

For better understanding and explanation, this paper is constructed in several sections. Section 2 summarizes the past studies regarding the use of NLP in SE. In addition to mapping their position in classified types, Section 2 explains the distinctive positioning of this paper. In Section 3, methodologies for doing this mapping study are described in a way that replicability is possible. Section 4 shows the result of this mapping study in the form of trends and charts with a clear explanation. Section 5 discusses the important findings for each RQ and presents points of view on NLP in SE. In Section 6, implication of some limitations in this study is discussed. Finally, Section 7 concludes.

Table 1
Comparison of this SMS with prior secondary studies.

Aspect	Prior Studies	This Study
Scope of Analysis	Focused on specific tasks (e.g., vulnerability detection [10,11], malware detection [12–14], code search [15,16])	Broad task coverage centered on understanding source code
Focus of Review	Often limited to modeling techniques or downstream applications (e.g., word embedding [17,18], deep learning [13,16,19–21])	Emphasizes the full NLP pipeline: preprocessing, embedding and modeling
Structure of Synthesis	Flat or topic-specific categorization (e.g., embedding [17,18], modeling [22])	Hierarchical based on types of task for each: embedding, preprocessing and modeling
Main Contribution	Survey/taxonomy of existing methods or tools (e.g., empirical study [23], trend & current knowledge [24,25]	Integrated map to guide future NLP research in software engineering

2. Related work

Numerous secondary studies have investigated the application of ML and NLP to source code. However, these works span diverse tasks, techniques and goals, ranging from code retrieval and summarization to vulnerability detection and defect prediction. To the best of our knowledge, no prior secondary study has offered a general, task-driven perspective on source code understanding using NLP. Most existing works focus on specific tasks or techniques rather than examining the area as a whole. As seen in Table 1, differences from several prior studies are shown, including the structure of this study. Having a full and hierarchical structure made this mapping study a complete map of the NLP workflow. To better position this SMS, we organize related work into four thematic clusters that align with the major task types identified in this study: detection tasks, searching and understanding tasks, summarization and generation tasks and foundational work.

2.0.1. Detection tasks

A substantial number of studies explore ML and NLP to improve software reliability through detection-oriented tasks. This detection includes vulnerability identification, malware classification, code smell recognition and defect prediction. For example, for vulnerability detection, study [10] conducted a systematic literature review on ML-based software vulnerability detection, aggregating model types, features and datasets, while [11] provides a road-map for selecting models in vulnerability detection systems. Another detection task of malware classification, such as using various static and dynamic features, was done by [12–14,21]. Detection tasks also extend to code quality issues like in [19], where they survey deep learning methods for detecting code smells, while [24] focuses on NLP-driven defect detection. Different tasks, from detecting defects to technical debt forecasting, are explored [23] and clone detection is reviewed in the context of deep learning [25]. These studies demonstrate the diversity of detection tasks and models, but often treat them independently. Unlike these fragmented views, our SMS provides a comparative synthesis across tasks and uncovers trends in more general aspects such as preprocessing, embeddings and modeling tailored for detection.

2.0.2. Searching and understanding tasks

Code search and understanding have been critical areas for applying NLP in software engineering. On the task of searching, [15,16] offer surveys on deep learning-based code search, highlighting the evolution from keyword search to semantic retrieval using dual encoders and pretrained models. On the task of understanding, [26] also touches on comprehension aspects within summarization. In addition, [27] discusses information retrieval-based techniques for understanding and annotating source code. While these works emphasize search or comprehension individually, they lack a unified view of how these tasks interact with preprocessing, embedding and modeling components. Our mapping contributes by categorizing how each stage of the NLP pipeline supports search and understanding.

2.0.3. Generation and summarization tasks

Tasks involving natural language output, such as summarization and code generation, are also noteworthy. For the generation side, [20] reviews deep learning models for code generation, while [17] focuses on the role of word embeddings in generative settings. On the summarization side, [26] provides a comprehensive survey of automatic source code summarization, including retrieval-based and neural approaches. While generation and summarization studies often emphasize model design or output quality, they less frequently discuss how task type impacts preprocessing needs or embedding strategies. Our SMS complements this by connecting model choices with task-specific design considerations across summarization and generation studies.

2.0.4. Foundational studies

Not based on any specific task, several studies provide foundational knowledge about how source code is represented and how pretrained models are developed or evaluated. Study [28] presents a systematic mapping study of code representation techniques, while [18] explores distributional semantics applied to programs. Furthermore, [22,29] examine pretrained model behavior and performance across various downstream tasks. Even more generally, [30,31] provide broader surveys of code intelligence, positioning pretrained models as central to modern NLP-based software engineering. These studies serve as conceptual groundwork. However, this SMS builds upon them in the source code understanding pipeline.

2.0.5. Positioning of this study

Unlike prior studies that focus narrowly on single tasks or technologies, this systematic mapping study is organized around source code understanding as a multi-layered process. It adopts a task-centric perspective, synthesizing relationships between task types (e.g., detection, generation), embedding methods, preprocessing techniques and machine learning models. It also highlights underexplored areas and identifies gaps for future work. Table 1 provides a high-level comparison between this study and representative prior reviews, where our main contribution is to guide future NLP research in software engineering.

3. Methodology

This study follows the guidelines for conducting systematic reviews in software engineering by Kitchenham et al. [7], guidelines for systematic mapping studies by Petersen et al. [8] and adopts the snowballing approach proposed by Wohlin [9] to ensure thorough and traceable identification of relevant primary studies. The methodology consists of five points: defining RQs, identifying studies, assessing study quality, extracting and synthesizing data.

3.1. Research questions

This study is guided by the following RQs, which reflect a hierarchical structure from the problem level (task) to implementation specifics (embedding, preprocessing and modeling). These questions are designed to capture a broad yet organized understanding of how NLP techniques are applied to source code:

- **RQ1:** What types of NLP-related tasks have been studied to support source code understanding?
  Rationale: Identifying the range of NLP tasks applied to source code helps characterize the current research landscape. It highlights which tasks are well-studied, which are emerging and where potential research gaps exist.
- **RQ2:** What preprocessing techniques and embedding methods are used for each task type?
  Rationale: Different tasks require different preprocessing steps and embedding. Analyzing these choices helps reveal common practices, evolving trends and underexplored techniques in how source code is prepared and encoded for NLP models.
- **RQ3:** Which machine learning models are used to address each task?
  Rationale: Examining the types of models used helps reveal which approaches are most common or effective for each task. It also shows how model choices are influenced by the nature of the task and code.
- **RQ4:** What research gaps exist in the application of NLP techniques to source code understanding?
  Rationale: Identifying research gaps helps guide future work by revealing underexplored areas and limitations in current approaches. This supports both academic progress and practical innovation in NLP for source code understanding.

3.2. Relevant study identification

To identify relevant studies, this mapping study primarily follows the systematic literature review (SLR) and SMS procedure [7,8], using a structured search protocol across multiple academic databases. In addition, forward and backward snowballing were conducted as a complementary step to enhance coverage and ensure that studies not captured through keyword-based queries were also considered, in line with the guideline [9].

3.2.1. Search strategy

The core search strategy was designed to retrieve peer-reviewed articles published between 2015 and 2024 (10-year span) that explore the use of NLP techniques for tasks related to source code understanding.

A combination of keyword-based search strings and Boolean operators was used. The creation started with four main parts: method, dataset, task and key topic. Hence, four "AND" operators will be used in the final keyword. For each main section, a few keyword candidates have been chosen as seen in Table 2. However, using all keywords can limit the search results. Therefore, selection was done starting with removing the task part. After discussion among the authors, having a task part that limits the task searched is counterintuitive to the objective of this study. Next is removing some synonym or keywords that have similar meaning, which are removing *"text vectorization"* which is similar to *"word embedding"* and removing *"software"* which is similar to *"source code"* but have less meaning in code as textual context. Not only removing, splitting the *"source code semantics"* and *"program semantics"* into their own part was done because having a semantic keywords is another parts of source code understanding.

After all of the iteration process that includes trial in the digital library and discussion among the authors, the final search string is:

Table 2
Keyword candidates for each four parts.

Method	Dataset
code embedding word embedding source code semantics program semantics text vectorization	source code programming language software
Task	**Key Topic**
representation interpretation prediction detection	understanding analysis

```
[''code embedding'' OR ''word embedding''] AND
(''source code semantics'' OR ''program semantics'') AND
(''source code'' OR ''programming language'') AND
(''understanding'' OR ''analysis'')
```

Searches were conducted across major digital libraries, including:

- IEEE Xplore
- ACM Digital Library
- Scopus
- SpringerLink
- ScienceDirect

A trial on the digital library of Web of Science was done, but it did not produce results. This initial step resulted in a total of 1278 candidate articles. Fig. 1 presents an overview of the study identification pipeline, illustrating the flow from database search to final study inclusion.

3.2.2. Cleaning and initial screening

The search results were first cleaned by removing duplicates and irrelevant entries, followed by a two-phase screening process:

- **Title and abstract screening:** Articles were filtered for relevance to source code understanding using NLP techniques.
- **Full-text review:** The remaining studies were examined in detail to assess their fit with the defined inclusion and exclusion criteria.

The inclusion and exclusion criteria used in the second phase screening process are as in Table 3. Finally, this cleaning process reduced the candidate pool to a refined **start set of 244 papers**.

3.2.3. Snowballing process

As a common practice in SMS, both backward and forward snowballing were applied. In this SMS, one cycle is applied in order to balance the coverage without introducing diminishing returns from repeated cycles to the already broad initial dataset. This additional step resulted in 18 more papers being added, bringing the total number of relevant studies included in the mapping to **262 papers**.

3.3. Study quality assessment

The quality assessment of candidate studies was done after the cleaning process during the study selection phase. This assessment reduces papers beyond the initial filtering. Each study was evaluated based on its capacity to address the RQs defined in this study. Specifically, six quality criteria were derived directly from the structure of the RQs:

1. Does the study utilize source code as the primary data?
2. Does the paper describe the task it aims to solve?
3. Is there a clear description of the embedding methods used?

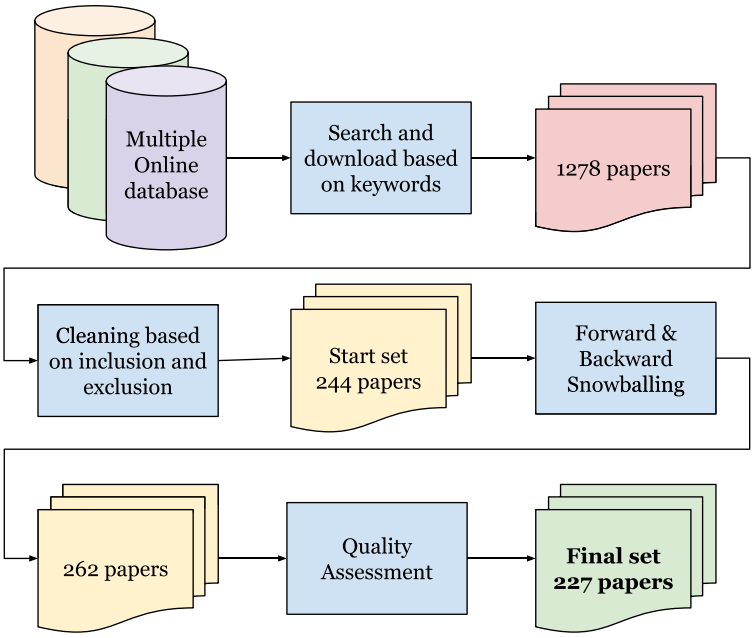


Fig. 1. General steps of the mapping study.

Table 3
Inclusion and Exclusion Criteria.

Inclusion Criteria	Exclusion Criteria
Study applies NLP or ML techniques to source code understanding tasks	Focuses on code metrics, developer behavior, or non-source code data
Published in a peer-reviewed journal or conference proceeding	Not peer-reviewed (e.g., preprints, theses, blog posts)
Written in English	Non-English language papers
Full text is accessible and analyzable	Duplicates or retracted publications
Addresses one or more tasks relevant to code comprehension, detection, generation, or search	Applies ML to unrelated software engineering areas (e.g., testing, deployment)
Present empirical methods, models, or experiments	A study including literature, mapping, or empirical study

4. Does the study explain its preprocessing techniques?
5. Is there a description of the machine learning model?
6. Does the paper include a performance evaluation or metric-based comparison?

This study introduces a lightweight quality assessment to manage the large number of studies retrieved and to maintain the objectivity of included studies. That decision is in line with guidelines for systematic mapping studies [8] where quality assessment is typically considered optional or non-standard. To balance inclusiveness with methodological rigor, a study was required to satisfy at least five of the six criteria. The exceptions were made for criteria related to preprocessing (criterion 4) or performance measurement (criterion 6), which could be answered "no" without disqualifying the study. This threshold ensured that only studies offering substantive methodological contributions or architectural insights relevant to the mapping were included. As a result of this process, **final set of 227 papers** met the required quality standards for subsequent analysis and synthesis. All identified and classified primary studies have been compiled into a dataset, which we have made openly available on Zenodo [32] to ensure transparency and reproducibility.

3.4. Data extraction

Data extraction was conducted based on the six quality assessment criteria, each of which corresponds to one or more of the RQs. For every included paper, relevant information was manually retrieved and recorded in a structured data extraction form. This process was carried out by the first author and validated by the second author to reduce bias and ensure consistency.

The extracted data included the following attributes:

1. **Source code usage:** Whether the study directly used source code as a primary data source and what programming languages or repositories were involved.
2. **Task description:** The specific NLP task addressed (e.g., vulnerability detection, summarization, code search). Tasks were then manually categorized into higher-level groupings to facilitate trend analysis.
3. **Embedding method:** Any representation technique used to convert source code into a form suitable for modeling (e.g., statistical embeddings, deep neural embeddings).
4. **Preprocessing approach:** Any procedures applied before embedding and/or modeling, including tokenization, filtering and normalization.
5. **Modeling technique:** The ML models applied to the task, including type (e.g., statistical, neural, hybrid) and architecture (e.g., Recurrent Neural Networks (RNNs), Transformers).
6. **Performance evaluation:** Whether the study reported any empirical evaluation, including metrics used (e.g., accuracy, F1-score, BLEU).

To ensure comparability across studies, all task names, model types and embedding techniques were normalized using a controlled vocabulary developed iteratively during the extraction process. Any ambiguous cases or non-standard terminology were discussed between the authors until consensus was reached. This structured approach enabled the construction of a comprehensive dataset that forms the basis for the synthesis and analysis presented in the following section.

3.5. Data synthesis

Following the data extraction phase, the collected information was compiled into a structured spreadsheet designed to support systematic synthesis. Each entry corresponded to a single primary study and included annotations for all key attributes derived from the RQs, such as the NLP task type, embedding method, preprocessing technique and ML model.

The synthesis process was guided by the hierarchical structure defined in this study: *Task, Embedding, Preprocessing and Model*. In addition, a controlled vocabulary was developed for each category, drawing from standard terminology in software engineering and AI research to ensure consistency. For example, tasks were classified into detection, search & understanding and generation & summarization, based on typical code understanding goals reported in the studies. Another example, Embedding methods were grouped by abstraction type (e.g., statistical, network-based, deep learning). The development of these terms was done through consensus meetings, where classification rules were discussed and refined iteratively. Terms were standardized early in the process to reduce ambiguity and ambiguous cases were documented and resolved by agreement. The analysis focused on identifying trends, distributions and co-occurrence patterns among these, with a particular emphasis on how different techniques are applied across various task types.

To facilitate pattern recognition and comparison across studies, the compiled data were normalized and grouped according to the categories established during the extraction phase. Descriptive statistics and frequency counts were used to quantify the prevalence of tasks, techniques and representations. These insights were then visualized using bar charts and distribution graphs to highlight dominant trends and areas of concentration. The results of this synthesis are presented and discussed in detail in the next section of this paper.

4. Results

The following are the results of this study that are split into four main results: general trend, task, embedding & preprocessing and modeling. These results reflect our findings in AI efforts in understanding source code in recent years, answering our RQs at hand.

4.1. General trend

The volume of research addressing source code understanding using NLP and ML has grown significantly in recent years. As shown in Fig. 2, the number of publications meeting our inclusion criteria increased from just one paper in 2015 [33] to 76 papers in 2024 (e.g., [34–36]. This upward trend became particularly pronounced starting in 2020, when the count rose from 13 publications to 32 in 2022, 61 in 2023 and peaked in 2024. This trajectory illustrates a growing interest in applying AI techniques to support various aspects of source code comprehension, indicating that the field is rapidly expanding and maturing as a research domain.

An important dimension of this study is the scope of programming language coverage within each research work. Figs. 3(a) and 3(b) illustrate how this aspect has evolved over time. As shown in Fig. 3(a), studies that evaluate techniques on multiple programming languages (e.g., C, C++, Java, PHP [37]) have steadily increased since 2020,

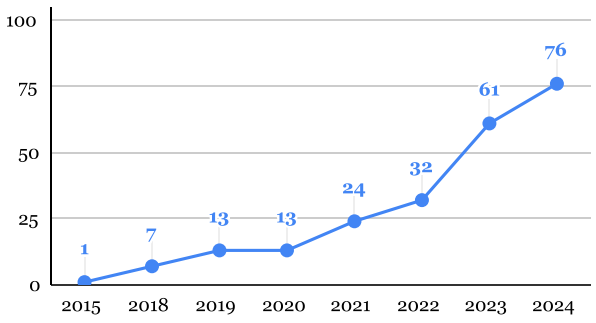
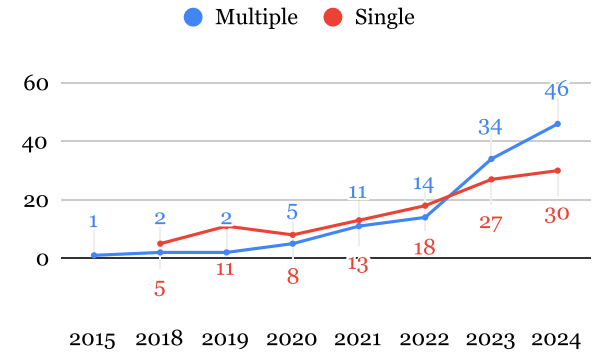
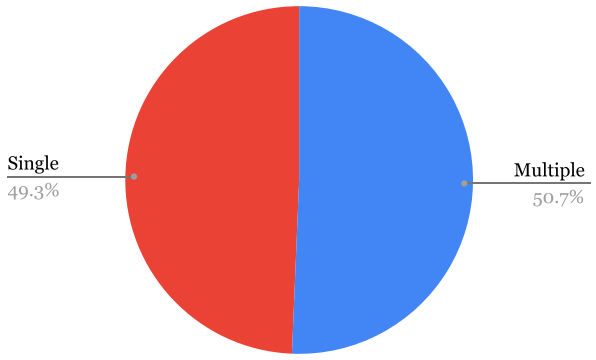


Fig. 2. Annual distribution of studies included in the mapping.



(a) Annual trend of studies using single vs. multiple programming languages



(b) Overall proportion of studies using single vs. multiple programming languages

Fig. 3. Distribution of programming language coverage.

surpassing single-language studies beginning in 2021 and maintaining that lead through 2023. This shift suggests a growing emphasis on the generalizability and robustness of source code understanding methods across language boundaries. The overall distribution (Fig. 3(b)) reflects a relatively balanced landscape: 50.7% of the studies use multiple languages, while 49.3% focus on a single language. This almost equal number indicates both the practical challenge of supporting diverse languages and the research community's growing commitment to addressing it.

4.2. Task categorization (RQ1)

To understand how NLP techniques are being applied to source code, we categorized the included studies into three high-level task groups: detection, search & understanding and generation & summarization. As shown in Fig. 4, detection tasks dominate the landscape, with a sharp increase beginning in 2020 and peaking at 62 studies

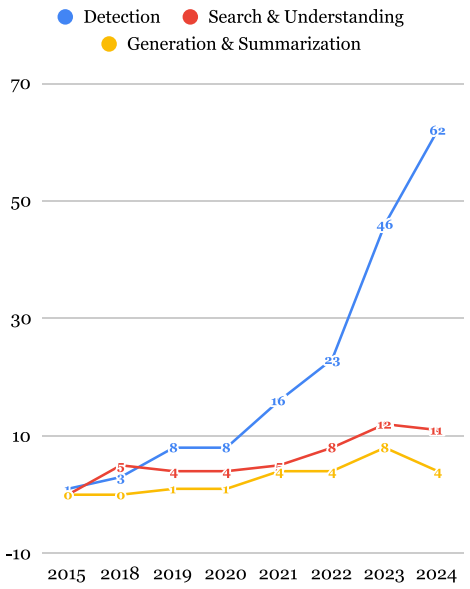


Fig. 4. Annual distribution of studies by task types.

in 2024. These tasks include vulnerability detection (e.g., [38–41]), malware analysis (e.g., [42,43]), code smell detection (e.g., [44,45]) and defect prediction (e.g., [46–48]), all of which are closely tied to improving software reliability and security. In contrast, tasks in the Search & Understanding category have shown slower but consistent growth, rising to 12 studies in 2023 and 11 studies in 2024. This includes studies focused on code search, traceability and comprehension. Meanwhile, Generation & Summarization tasks such as code summarization and automated code generation remain the least explored, though they have maintained a modest presence since 2019.

4.2.1. Detection subtasks

Detection tasks in source code understanding can be broadly categorized into two main subtypes: entity detection and similarity detection. As shown in Fig. 5(a), entity detection dominates the landscape, encompassing studies on vulnerability detection, malware analysis, defect prediction and code smell detection. These tasks share a common objective: to identify specific, semantically meaningful entities embedded within the source code. Their popularity can be attributed to both practical relevance, especially in the domains of software quality and security and the relative abundance of labeled datasets that facilitate supervised learning approaches. In contrast, similarity detection tasks, mostly represented by code clone detection, aim to identify structural or semantic redundancies in code. Although fewer in number, these studies address foundational challenges in code maintainability and refactoring, which are essential for large-scale software evolution. The overall distribution underscores the community's prioritization of security and correctness through entity detection, while also acknowledging the enduring importance of similarity analysis.

Detection tasks form the largest group of studies in this mapping and exhibit substantial internal variation. As shown in Table 4, most detection studies fall under the entity detection category, with vulnerability detection as the most frequent subtask with 83 studies. This reflects the critical importance of security and reliability in modern software systems and the active role of AI in identifying known and unknown weaknesses. Defect & bug prediction is also highly represented, with 33 studies indicating continued efforts to improve software quality and reduce maintenance costs. Followed by various classification tasks, such as API classification and Commit classification, which appear less frequently but remain relevant for specific applications, with a total of 25 studies. Under the entity similarity category, code clone and code similarity detection consist of 19 and 12 studies, respectively. The rest of the smaller part of the subtask consists of malicious code detection, inference & typing, detecting code smell & code quality and some other entity detection tasks (e.g., parallelism detection and maintainability prediction). This distribution suggests that while entity-level concerns remain dominant, structural similarity detection continues to hold relevance in scaling and maintaining large codebases.

4.2.2. Search & understanding subtasks

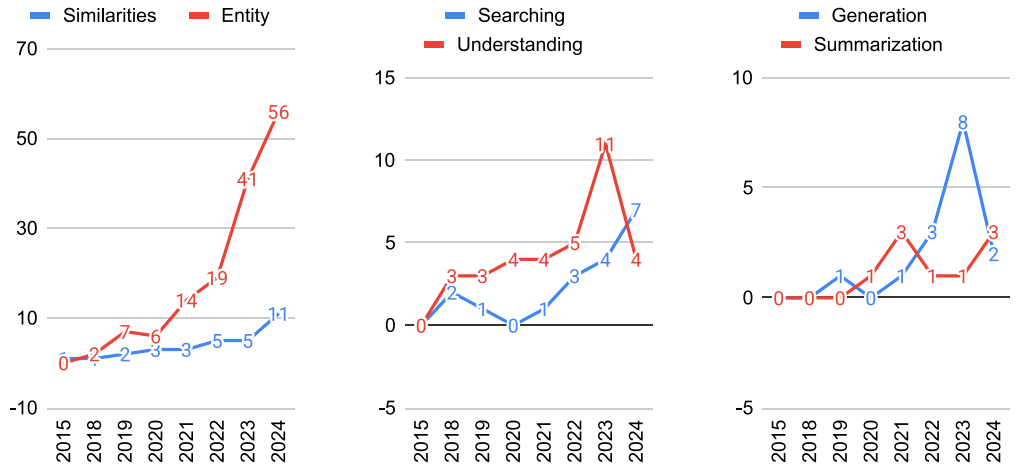
Search & understanding tasks represent an important, though less dominant, segment of the research landscape in source code understanding. As illustrated in Fig. 5(b), the volume of studies addressing search-based tasks has grown steadily since 2020, with a particularly notable increase between 2022 and 2023, but with a shocking downfall in 2024 for understanding tasks. These tasks typically focus on retrieving relevant code artifacts based on natural language queries or structured inputs and they play a crucial role in developer productivity tools. In contrast, understanding tasks that include program comprehension, traceability link recovery and context-aware exploration have seen more modest growth. This may reflect the complex and less standardized nature of understanding research, which often requires fine reasoning and deeper semantic modeling. Despite their smaller number, understanding tasks contributes meaningfully to the goal of human-centric code intelligence. The overall distribution suggests that while search tasks have benefited from recent advances in semantic representation and pre-trained code models, understanding tasks remain an emerging and promising area with plenty of room for innovation.

Search & understanding tasks represent a growing and strategically important area within AI-assisted source code comprehension. According to Table 5, the majority of studies in this category focus on code search and retrieval, with 21 studies. These works aim to bridge natural language queries and relevant code segments, supporting use cases such as semantic code search, issue linking and trace recovery. Meanwhile, the understanding branch includes localization with 12 studies, embedding & representation (a critical task for base code understanding) with 6 studies and a broader group categorized as normalization, refactoring and other understanding (e.g., API mapping and detecting comment scope) with 4 studies on average. These studies encompass efforts to model developer intent, identify code purpose and assist in navigation or summarization based on code context. While the number of studies in this group is smaller compared to detection, their diversity and conceptual depth reflect an expanding focus on human-centric code comprehension.

4.2.3. Generation & summarization subtasks

Although representing the smallest segment among the task types, generation & summarization tasks have seen consistent research interest over the past five years. As illustrated in Fig. 5(c), summarization tasks have maintained a steady presence since 2020, with a noticeable increase in 2021 and 2024. These tasks aim to generate concise, human-readable descriptions of code snippets or functions and are increasingly important in tools for improving code comprehension and documentation. In contrast, code generation tasks, focused on producing source code from natural language or partial input, have emerged more recently and remain less frequent, likely due to the higher modeling complexity. The relatively small number of generation and summarization studies compared to detection or search tasks highlights both the challenges and the untapped potential of these tasks, particularly in advancing human-AI collaboration in software development.

Generation and summarization tasks, though fewer in number compared to detection and search-related efforts, reflect a steadily emerging focus on enhancing human-AI collaboration in software engineering. As outlined in Table 6, most studies in this category address code summarization, with 9 studies focusing on producing concise natural language descriptions of code behavior. The subtask of generation



(a) Detection (b) Search & Understanding (c) Generation & Summarization

Fig. 5. Annual distribution of studies by subtask types.

Table 4
Included studies by Detection subtask.

Detection	Relevant Studies
Vulnerability Detection (83)	[34–37,39–41,49–61,61–119] **[120–123]**
Defect & Bug Prediction (33)	[46–48,124–140] **[120,123,141–151]**
Classification Tasks (25)	[38,152–162] **[62,121,122,143,145–148,163–167]**
Clone Detection (19)	[33,168–174] **[122,143,144,147,149,150,163–165,175,176]**
Similarity Detection (12)	[177–184] **[141,142,146,175]**
Malicious Code Detection (5)	[42,185–187] **[43]**
Inference & Typing (5)	[188–191] **[62]**
Code Smell & Quality (3)	[44,45] **[120]**
Other Entity Detection (2)	[192,193]

Note: Studies highlighted in **bold** are assigned to more than one subtask.

Table 5
Included studies by Search & Understanding subtask.

Search & Understanding	Relevant Studies
Code Search & Retrieval (21)	[194–208] **[144,150,175,176,209,210]**
Bug & Vulnerability Localization (12)	[211–220] **[43,166]**
Code Embedding & Representation (6)	[221–224] **[209,225]**
Testing, Mutation & Repair (5)	[226] **[227–230]**
Identifier & Normalization (4)	[231,232] **[175,233]**
Refactoring & Optimization (4)	[234,235] **[225,233]**
Other Understanding (4)	[236–239]

Note: Studies highlighted in **bold** are assigned to more than one subtask.

consists of several subtasks, such as program repair & error fixing, comment, commit & change and other generation (e.g., API completion and recommendation) subtasks, with an average of 3 studies. Overall, while this task group remains less populated, its methodological complexity and practical relevance suggest significant room for innovation and impact in future work.

Summary of RQ 1 findings: Detection tasks dominate the field, particularly vulnerability and defect detection. Search and understanding tasks are growing but remain less frequent, while generation and summarization tasks are still underexplored.

4.3. Preprocessing and embedding (RQ2)

To better understand how source code is represented and prepared for NLP-based modeling, this section presents an analysis of both preprocessing techniques and embedding methods across the three task types. Preprocessing and embedding are discussed individually to see the general trends and the overall methodological choices.

4.3.1. Preprocessing

Preprocessing techniques play a foundational role in preparing source code for embedding and modeling. In this study, preprocessing is broadly grouped into three categories: split or structural processing, removal or cleaning and replacement or normalization. Each category is non-restrictive and produces good distinctions between them.

Split or structural processing is the process to change the usually large and unstructured text into a smaller and more structured form. This includes operations such as tokenization, identifier splitting (e.g., turning camelCase into camel and case) and abstract syntax tree (AST) flattening (changing the text into tree form). Split-based preprocessing has consistently been the most widely adopted approach, as illustrated in Fig. 6, where it has consistent growth from only one study in 2015 to 64 in 2024. Accounting for 170 studies overall

Table 6
Included studies by Generation & Summarization subtask.

Generation & Summarization	Relevant Studies
Code Summarization (9)	[177,240–244] **[210,245,246]**
Program Repair & Error Fixing (5)	[247,248] **[151,227,229]**
Comment Generation (5)	[249–251] **[245,246]**
Commit Message & Change Generation (3)	[252] **[141,167]**
Code Translation & Generation (3)	**[149,150,176]**
Mutation & Adversarial Generation (2)	**[228,253]**
Other Generation (2)	[254,255]

Note: Studies highlighted in **bold** are assigned to more than one subtask.

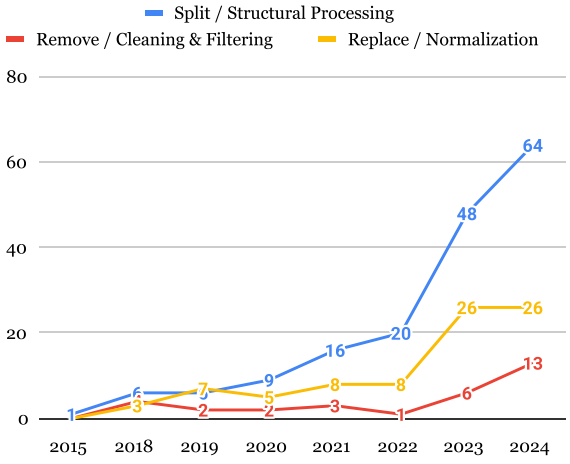


Fig. 6. Annual distribution of preprocessing techniques in general categories.

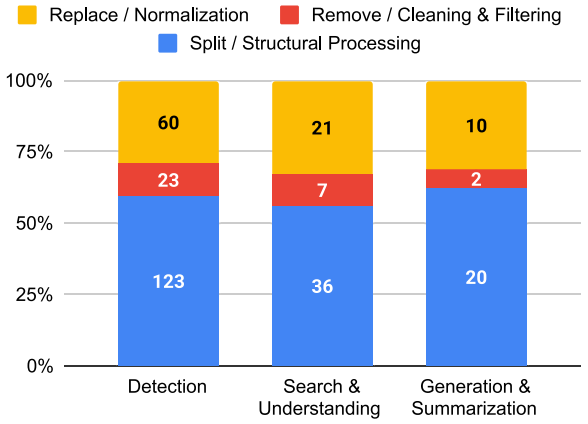


Fig. 7. Mapping of preprocessing technique categories accross task types.

illustrates its foundational role across all tasks and modeling strategies. These methods retain structural integrity, making them essential for syntax-aware models and embeddings.

As explained by the name, the removal, cleaning, or filtering process was done by reducing the number of texts to be used for further use. As seen in Fig. 6, while less frequent overall (31 studies), this process is still there to address code cleanliness by filtering out redundant or noisy elements, such as comments, import statements and unused code. Its use is generally selective, with modest growth over time (e.g., six studies in 2023, 13 in 2024). These methods are especially relevant in tasks where extraneous tokens could distract from semantic content, such as code retrieval or traceability.

Replace or normalization works by changing the current natural text that has many variations into a single representation. A total of 83 studies, as observed in Fig. 6, have gained momentum in recent years. These include variable name anonymization [120,168,177], casing normalization [214] and standardization of literals [49] designed to abstract code into model-friendly formats. Usage rose notably after 2022, peaking at 26 studies in 2023 and in 2024. This trend reflects growing attention to model generalizability, particularly in detection and generation tasks where overfitting to identifier names can be significant. However, while replacement can reduce overfitting and help models generalize, it may also obscure meaningful semantic signals, which explains its more selective use, especially in generative or comprehension tasks where identifier names carry intent.

The distribution of preprocessing techniques varies across task types but remains in a similar proportion, reflecting similar priorities in how code is prepared for analysis. As shown in Fig. 7, detection tasks use split/structural processing most extensively, with 123 studies. This heavy use of split-based techniques underscores the importance of structural clarity in identifying defects, vulnerabilities, or smells. In search and understanding tasks, replacement processes are used less in number than detection, 36 studies, but more in generation & summarization of 20 studies. However, if seen by the percentage, this task uses more replacement or normalization compared to the other two tasks. This likely reflects the need to preserve natural identifiers and comments that can support alignment with natural language queries or traceability links. Generation & summarization tasks apply preprocessing more selectively, with only two removal processes recorded in [242,246]. The rarity of the removal process happens because this task often depends on preserving stylistic and lexical features of the original code. As such, cleaning is minimized to retain natural code semantics and improve output fluency.

4.3.2. Embedding

The evolution of embedding methods across all tasks in source code understanding reveals a clear and accelerating shift toward deep learning–based representations. As shown in Fig. 8, statistical and network-based embeddings were dominant in the early years, with steady use through 2019. Statistical methods, often grounded in token frequency [67] or n-gram representations [125], showed early promise due to their simplicity and compatibility with traditional models. Network-based embeddings, including graph and tree-based techniques, also gained traction during this period, especially in structurally focused tasks like defect prediction [144,146,147] and clone detection [33,168,169]. However, starting in 2020, deep learning embeddings began to increase sharply, overtaking both other categories by 2022 and peaking in 2024 with over 50 studies adopting these methods. This growth is primarily driven by the adoption of transformer-based architectures [120,175,194] and pre-trained models [202], which offer greater capacity for capturing semantic and contextual relationships within source code. The trend reflects the broader shift in software engineering research toward learned, task-specific representations that

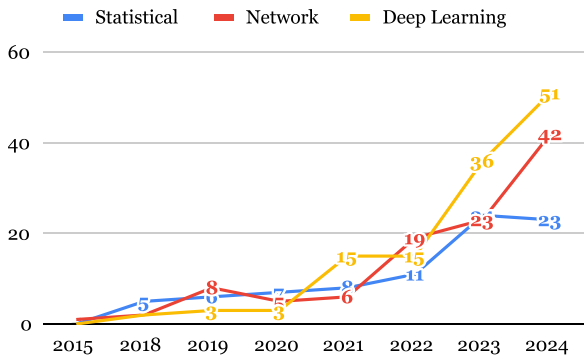


Fig. 8. Annual distribution of embedding methods in general categories.

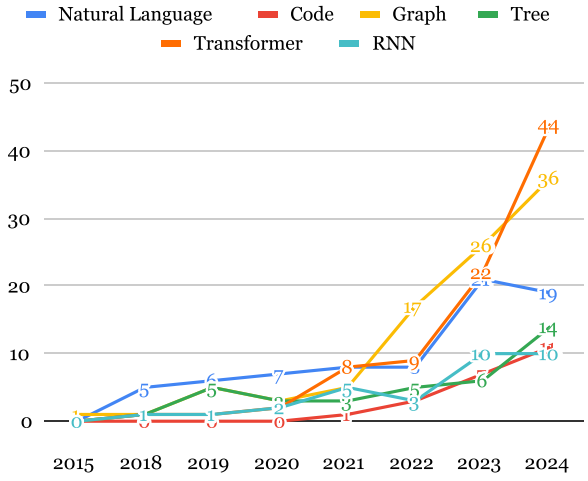


Fig. 9. Annual distribution of embedding methods in specific subcategories.

can generalize across varying code structures and languages. This transition underscores the field's convergence on neural embeddings as the standard foundation for intelligent code analysis.

A detailed analysis of embedding subcategories across all task types underscores the field's transition toward deep learning representations. As shown in Fig. 9, transformer-based embeddings have shown the highest rise, increasing from just one study in 2018 to 44 studies in 2024 and reaching a cumulative total of 87. Their scalability and effectiveness in both generative and discriminative tasks make them the most widely used embedding subclass overall. Graph-based embeddings are the most common and have grown substantially, from one study in 2015 to 36 in 2024, totaling 94 studies, highlighting their relevance in modeling syntactic structures through abstract syntax trees and parse trees. Tree-based embeddings, while less frequent overall (37 studies), show a clear upward trend, particularly in the last two years, increasing from six in 2023 to 14 in 2024, reflecting their value in representing data and control flow. RNNs, once more prevalent in early neural approaches, peaked in 2023 with 10 studies and maintained the same count in 2024, suggesting that they remain in limited use, primarily in legacy or lower-resource settings. Statistical embeddings, including natural language–based (74 studies total) and code token–based methods, show relatively modest growth and are now primarily used in either early work or as baseline comparisons. Overall, the evidence reveals that transformer and tree-based embeddings now form the core of source code representation research, while statistical and earlier neural models like RNNs have largely decreased.

This overall trend toward learned embeddings becomes even clearer when broken down by task. As shown in Fig. 10, deep learning embeddings dominate across all task categories, though the degree of

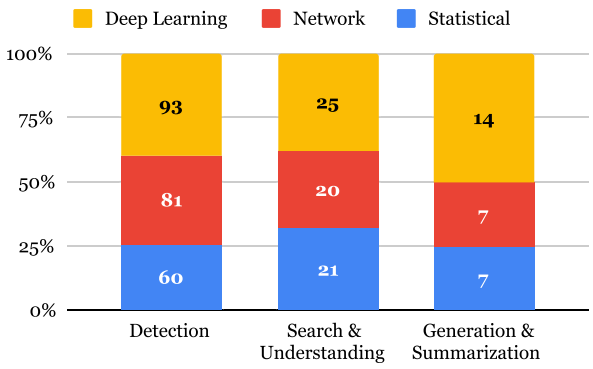


Fig. 10. Mapping of embedding method categories accross task types.

adoption varies. In detection tasks, 93 studies employed deep learning–based embeddings more than the number using network-based (81) and statistical embeddings (60). The high number of deep learning and graph embedding in the mature detection task indicates that the detection task is the one leading the trend shift. In search and understanding tasks, deep learning is the preferred method, applied in 25 studies compared to 20 using network and statistical embeddings. This suggests the importance of embeddings that can capture semantic relationships and support cross-modal retrieval, particularly between natural language and code. Generation and summarization tasks exhibit the most dependence on deep learning, with 14 studies out of 28, which is exactly 50% of all. Only 7 studies in this category used statistical methods, likely as support to other embedding methods [245,250,251,254]. These findings reinforce the conclusion that across all task categories, deep learning and transformer-based models have become the default representation strategy for source code understanding.

Summary of RQ 2 findings: Preprocessing remains task-dependent: split-based techniques are consistently applied, while replacement and removal methods vary, reflecting differences in structural vs. semantic preservation needs. Deep learning–based embeddings, especially transformer models, have become dominant across all tasks, replacing statistical and network-based approaches.

4.4. Modeling (RQ3)

Modeling is the final and decisive stage in the source code understanding pipeline, where structured or embedded representations are mapped to task-specific outputs. The choice of modeling techniques is closely intertwined with the type of task, the nature of the input representation and the complexity of the prediction or generation objective. In this section, we analyze the trends in statistical models, traditional machine learning algorithms and neural networks across the three task types. We also examine the specific subcategories of neural architectures employed, such as transformers, recurrent networks and graph-based models, to capture how architectural innovations have shaped the evolution of this research domain.

4.4.1. Modeling for detection

Modeling techniques used in detection tasks have undergone a transition from classical machine learning to deep learning–based approaches. As shown in Fig. 11, traditional machine learning models such as decision trees [123] and support vector machines [155] were common prior to 2020, often paired with handcrafted features or statistical embeddings. However, starting in 2021, neural network–based models have rapidly increased, overtaking traditional methods and becoming dominant. This shift is driven by the growing complexity of detection tasks, particularly in security and quality assurance domains.

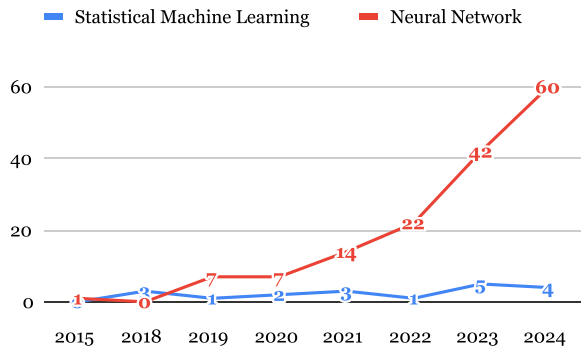


Fig. 11. Trend of modeling methods used in detection tasks.

Further insight into model architecture choices is provided in Fig. 12, which breaks down neural networks by subclass. Model usage in detection tasks reveals a strong shift toward deep learning or neural networks, as explained before, particularly graph-based and transformer-based neural architectures. Graph neural networks (GNNs) lead with 74 studies (e.g., XShellGNN [186], Adaptive GNN [62] and Tensor GNN [83]), reflecting their strength in capturing control and data flow dependencies, which are essential in identifying vulnerabilities and structural anomalies. Transformer networks (e.g., TransDPR [152] and PDBert [53]) also play a major role (65 studies), often paired with large-scale pretraining for defect or smell detection. Traditional architectures such as recurrent neural networks (19), convolutional neural networks (18) and even generative adversarial networks (3) appear in select studies, each suited to specific problem formulations. Interestingly, classical models such as linear models (e.g., TF-IDF [226] and Support Vector Machine (SVM) [155])and tree-based approaches (e.g., Decision Tree [162] and Tree Graph Gated Attention [68]) remain in use, particularly in earlier work or where interpretability and low complexity are prioritized. This distribution demonstrates a diverse modeling landscape, with an increase of deep neural models tailored for complex detection tasks.

4.4.2. Modeling for search & understanding

The modeling for search & understanding tasks reflects a shift toward deep learning approaches, but with a slower uptake compared to detection. As shown in Fig. 13, traditional machine learning techniques remained in use through 2021, but were gradually beaten by neural models in the following years. This transition aligns with the increasing complexity of semantic retrieval and comprehension tasks, which demand models capable of aligning natural language with code semantics and capturing context beyond lexical similarity.

The underlying architectures employed in these tasks further illustrate this shift. As indicated in Fig. 12, transformer-based models are now predominant, particularly in code search and traceability applications where sequence modeling [227] and cross-modal [197] understanding are essential. Graph-based models appear less frequently but remain relevant in scenarios involving structural alignment or context-aware comprehension [164]. Recurrent networks, while still in use, have declined in favor of transformers due to their scalability and superior performance in representing long-range dependencies. The overall trend reveals a growing reliance on semantically aware models to support the reasoning required for search and understanding in software engineering.

4.4.3. Modeling for generation & summarization

Generation & summarization tasks in source code understanding have used neural modeling approaches almost exclusively in recent years. As illustrated in Fig. 14, deep learning models have dominated since 2021, with minimal reliance on traditional machine learning. This reflects the demands of generative tasks, which often require end-to-end modeling of language and structure in both code and natural language. Classical models, which depend heavily on feature engineering, are largely unsuitable for the complexity of these tasks, though there is an anomaly in 2023.

In generation & summarization tasks, transformer networks are the predominant modeling choice. As shown in Fig. 12, transformer with 12 studies has a higher percentage than any other task class. This aligns with the widespread success of transformers in sequence generation tasks [253], where fluency, context preservation and abstraction are critical. Other neural models appear less frequently: general neural networks (7), GNNs (3) and RNNs (2) suggest that alternative approaches have been explored but are less favored, likely due to limitations in handling long-range dependencies. Classical methods like linear models (1) and tree-based models (0) are nearly absent, indicating that traditional ML techniques are not suited to the generative demands of these tasks. Overall, the modeling landscape in this task group reflects a dominance around transformer-based architectures, reinforcing their role in modern code-to-text and text-to-code generation systems.

Summary of RQ 3 findings: Neural network models, particularly transformers, have overtaken traditional machine learning approaches in all task categories. Detection tasks increasingly adopt graph-based and deep learning models, while generation tasks rely almost exclusively on transformer architectures.

5. Discussion

The following discussion interprets the results of this study in relation to the four RQs, providing a task-structured synthesis that connects the observed trends with their implications. Each subsection discusses the findings from Section 4 and places them in the broader context of AI research on source code.

5.1. RQ1: Task categorization

The task distribution shows a clear dominance of detection studies, followed by a modest presence of search and understanding and limited work on generation and summarization. The results in Section 4.2 and Fig. 4 reveal a strong concentration of research on detection tasks. The consistency of these results over time (Fig. 4) reflects both research maturity and practical demand.

5.1.1. Detection

Detection remains a dominant focus, driven by the availability of well-labeled datasets and the practical demands of ensuring software security and quality. One of the most used datasets is SARD (Software Assurance Reference Dataset), which was used in 23 studies about Vulnerability (e.g., [39,49,50]). There was also Big-Vul with 16 studies (e.g., [53,59,62]), Devign with 15 studies (e.g., [40,73,144]) and ReVeal with 16 Studies (e.g., [40,50,53]) where all of them are for vulnerability detection. There are some other datasets that are used multiple times as well, such as BigCloneBench for clone detection, PROMISE for defect detection and FFmped & QEMU for vulnerability detection. Though most use the available datasets, a few choose to produce their own custom dataset. For example, datasets mined from GitHub with 27 studies (e.g., [49,51,196] and some custom datasets (e.g., [67,74,86]. These big variations of datasets regarding detection really show that this task is mature and can be applied to supply practical demand.

On the other hand, there are signs of a stagnant research direction: detection may be reaching a plateau in innovation because many recent studies revisit the same sub-problem (e.g., vulnerability detection) with slightly modified models or embedding. Furthermore, those modification is used only to gain one digit percentage of performance measurement in the same dataset (e.g., for SARD [35,94,97,103], ranging from 92% to 95% in F1-Measure). This will make future innovation more challenging because current performance is close to 100% and that will reduce the amount of research in that topic.

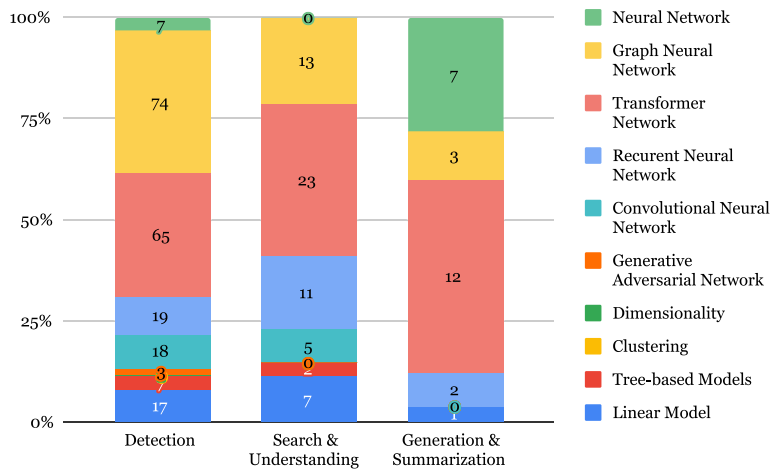


Fig. 12. Distribution of neural network subclasses used in the included studies.

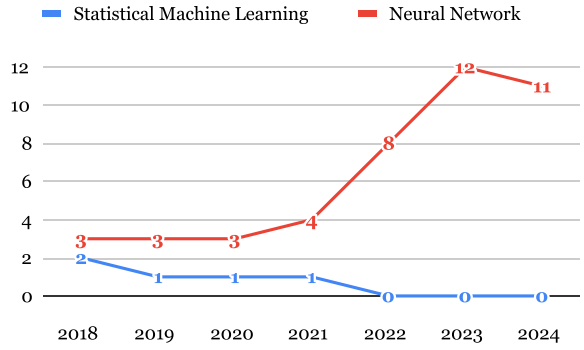


Fig. 13. Trend of modeling approaches used in search & understanding tasks.

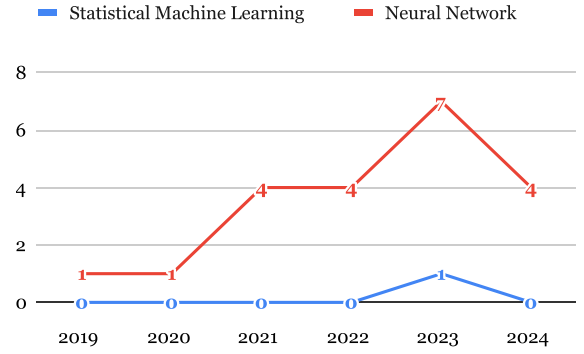


Fig. 14. Trend of modeling approaches used in generation & summarization tasks.

5.1.2. Search & understanding

In contrast, search & understanding tasks remain relatively underexplored, although they have shown steady growth in recent years. Code search, in particular, continues to rely on structurally shallow approaches, focusing mainly on matching user queries with relevant code snippets. A review of recent studies [197,205,206] shows that while newer models incorporate advanced techniques such as contrastive learning, multimodal representations and ensemble methods, they primarily aim to improve retrieval performance through refined similarity learning. These methods often enhance token-level or representation-level matching but still struggle to capture deeper code semantics, such as program behavior, execution flow, or developer intent. As a result, despite technical advances, the fundamental approach to code search remains largely unchanged, reflecting incremental rather than transformative progress.

Understanding tasks is broader and more fragmented. Unlike detection or search, "understanding" encompasses a range of goals. Several studies target foundational comprehension, such as code representation [221] and identifying functionality from code or linking methods to documentation [222]. These tasks often serve as proxies for evaluating embedding quality and interpretability. Other understanding tasks are context-specific, such as program repair [227] and design github repository recommendation [237]. This understanding task mapping shows that it grows wider rather than higher, giving more room to explore different understanding objectives.

5.1.3. Generation and summarization

Generation and summarization are among the most difficult problems in source code understanding. They require the model to not only comprehend but also synthesize meaningful outputs in either natural language or executable code. These tasks remain relatively immature compared to detection. Many studies are proof-of-concept with limited datasets. Methods such as Fret [246] and EditSum [241] introduce novel approaches but evaluate their performance on non-standard or small-scale datasets. Furthermore, evaluation standards are still inconsistent, such as BLEU, ROUGE and METEOR are used together (e.g., in [240,243,246,252], but some still use precision recall (e.g., in [176,251]). Adding the still-not-stable research, another turn was made in this task, which is the emergence of LLMs. Methods of LLMs such as CodeT5 used in [254] open new frontiers for generation. Though the trend in the past seemed immature, it can become a solid research foundation.

5.2. RQ2: Preprocessing and embedding

Deep learning has become the dominant approach even for preprocessing and embeddings across tasks. Its ability to capture higher levels of abstraction and semantics makes it especially valuable in generation and detection tasks. Preprocessing techniques are widely applied but often receive little attention in terms of their rationale or design choices. While graph-based embeddings continue to play an important role, traditional statistical embeddings have steadily fallen out of favor. The following discussion explores these trends and considers their broader implications for future research.

5.2.1. Preprocessing

Despite its commonness, preprocessing receives little detailed explanation in most studies. Techniques are typically adopted without

much justification and their impact on model performance is rarely isolated or ablated. In addition, most preprocessing pipelines still rely on established methods such as tokenization, identifier splitting and AST flattening. As seen in Fig. 7, 181 studies used split-based techniques, yet very few described how they selected or tuned these processes (e.g., [196] gives a complete list of preprocessing). Meanwhile, removal methods (e.g., stopword or comment deletion) are often undocumented regarding their removal list (e.g., [177]). This is unfortunate as preprocessing is the front door to the NLP and it clearly affects the repeatability and the performance of the result.

To make things more complicated, some recent studies have started to treat preprocessing not as a static stage but as part of the learning pipeline. For example, transformer-based models benefit from context-preserving preprocessing strategies. Instead of anonymizing identifiers, several summarization and search studies retain naming structures to preserve semantic signals. For instance, [157] shows improved performance when retaining camelCase identifiers for summarization tasks. In deep learning pipelines, especially those using sequence-to-sequence models, preprocessing is increasingly shaped by token-level embedding requirements. Byte-pair encoding (BPE) [42] and position-aware encoding [134] are examples to align input representations with pretrained model vocabularies. While these trends reflect better integration between representation and model architecture, they are often undocumented and left to default implementations. At worst, they may be treated as not using preprocessing at all, though it is invisible.

5.2.2. Embedding

Deep learning embeddings now form the backbone of source code representation. Because the embedding process is tightly related to the model, the rise in the use of deep learning naturally leads to the rise in deep learning embedding. Across tasks, transformer-based models such as CodeBERT [69,197] and GraphCodeBERT [202] are increasingly favored due to their ability to model long-range dependencies, contextual relationships and multilingual token distributions. Although this may seem like a natural progression in model complexity, it also introduces new challenges. Deep learning embeddings often function as black boxes, making it difficult to interpret what aspects of the code are being captured. Moreover, these models typically require large amounts of training data and computational resources, which can limit their accessibility and reproducibility. As the field continues to embrace deep learning, a deeper understanding of how embeddings influence downstream tasks will be essential for building more transparent and efficient systems.

The fall of statistical embeddings reflects the broader shift toward deep learning approaches. Because embeddings are closely tied to modeling strategies, the decline in statistical machine learning methods has naturally led to a reduced use of statistical embeddings as well. Techniques such as n-grams, TF-IDF and doc2vec, which were widely used in earlier work, now appear mostly as baselines [88,226,237]. Interestingly, a study [153] still explores combining statistical features with deep learning models, suggesting that while statistical embeddings are no longer at the forefront, they may still play a complementary role in certain contexts. As deep learning continues to dominate, understanding when and how to integrate these traditional approaches could offer valuable insights for building more effective and interpretable models.

Graph-based embeddings hold a stable position in the current landscape. While they are neither the most widely used nor the fastest growing, they remain a common choice. This persistence is largely due to the structured nature of source code, which contains elements such as loops, branches and scope boundaries. Graph embeddings are well-suited to capturing this structure, allowing models to learn from both the syntax and the underlying logic of programs. For example, [242] in summarization, [219] in understanding of bug localization and [54] in vulnerability detection. Overall, this trend suggests that network-based embeddings have reached a level of maturity and stability that makes them a promising option for future work.

5.3. RQ3: Modeling

The evolution of modeling approaches in source code understanding reflects the field's transition from conventional statistical machine learning to increasingly specialized neural architectures. As seen from Figs. 11, 13 and 14, neural network machine learning is clearly replacing statistical machine learning. Early studies frequently employed statistical models such as linear classifiers, particularly in detection tasks. However, as task complexity and representational depth increased, these models were gradually displaced by deep learning architectures. Neural networks, especially transformer-based and graph-based models, have become the dominant modeling paradigm across all task categories. The choice of model is closely aligned with the nature of the task: structural models like GNNs (e.g., [178]) are favored in detection, attention-based models like transformers lead in generation and summarization. These evolutionary shifts can be clearly seen and discussed in this section.

5.3.1. Statistical machine learning

Statistical models such as linear classifiers and decision trees are not the future. They appear in less than 10% of all studies, primarily in early detection-focused papers. These models persist in part due to their simplicity for a proof-of-concept research. For example, [183] created an evaluation on representation learning where the main focus in the embedding. Even so, a full NLP pipeline is still needed to get the result; they use Logistic Regression, Decision Tree and Naive Bayes as base models to compare the effect of various embeddings. Other than that, solving simple classification problems in detection tasks, such as Self-Admitted Technical Debt identification [155] using SVM, still produces a good performance (82% in accuracy). However, for tasks in search and understanding, statistical models have been virtually abandoned. For example, in [199] where they use Neural Code Search algorithm that uses TF-IDF and they achieve 68% in accuracy. The need to align natural language with code and to capture semantic intent exceeds the representational power of these techniques. On summarization and generation tasks, there is a demand for context-aware, sequence-level modeling, something that statistical models cannot handle. No included study uses a purely statistical model for these tasks; study [255] uses a combination of statistical and evolutionary models to generate API recommendations, further confirming their obsolescence in generative settings. From the findings, nearly no recent research uses pure statistical machine learning to do source code understanding to solve real-world problems, which makes it hard to say that future use is needed.

5.3.2. Neural network

Neural network-based models are the current state-of-the-art machine learning models for source code understanding. It includes any network-based model, such as graph, tree, neural and transformer. The increasing trend affects all of them but only some, as seen in Fig. 12, are creating the big impact.

Graph-based models have proven particularly effective in detection tasks. As shown in Fig. 12, GNN is the most used model. GNN is mimicking semantic relationships through representations like control flow graphs (CFGs) and ASTs. For example, [114] uses GNNs over data flow graphs to capture vulnerability signatures and achieve 89.5% in accuracy and [99] uses GNN with multi-view fusion to detect vulnerability, achieving 93.2% in accuracy. In many cases, graph or tree structures are not models themselves but act as intermediate representations. These are then passed into feedforward, recurrent, or transformer layers for downstream classification, effectively decoupling structure encoding from learning. For example, using Bilateral Dependency Neural Network with AST-based embedding [153] successfully classifies algorithms with an accuracy of 93% on a single language and 86% on multiple languages. If seen as a full pipeline, the use of a graph or tree structure in the model is helpful in achieving high performance.

For search & understanding and summarization & generation tasks, the transformer-based model is the champion. Transformers bypass the need for explicit graph connections by using self-attention to capture global context. This architecture is especially dominant in generation & summarization tasks, where fluency and long-range dependencies are critical. Studies like [194] demonstrate how transformer-based models outperform Convolutional Neural Networks in code search. Study [246] solve code summarization using Functional Reinforced Transformer and outperform Multi-way Tree-LSTM by +14.23% in BLEU-4 score. This better performance measurement gives confidence in the transformer-based model to solve these tasks.

Other than the big two models of graph-based and transformer-based, there are machine learning models used for code understanding and achieving good performance. One of them is an RNN that was once widely used in NLP. However, their sequential limitations and difficulty in modeling long-term dependencies have led to their decline in source code understanding. For example, study [51] that detects vulnerability in source code achieves an accuracy of 92% for RNN and LSTM, 94% for transformer and 95% for the ensemble of them all. From that sample, it is seen that RNN by itself is losing to transformer, but still gives a good improvement when used together as an ensemble. Though it is not the best model, RNN models still have an impact on source code understanding. Another is Convolutional Neural Networks (CNN), which can be unconventional in NLP use. A model of CNN appears in a few studies, such as [125], which achieves 78% in F1-measure and [55] 87.7% in accuracy for the detection task. Generative Adversarial Network (GAN), also used in [126] for defect detection, performs poorly with only 32% in F1-score and 69% in recall. This poor performance of the rarely used model shows why it is rare to use them.

5.3.3. Performance measurement

Throughout the development of artificial intelligence in source code understanding, the commonly used performance measurement is still the staple. An old measurement of accuracy remains a staple metric, particularly in classification-heavy detection tasks. However, accuracy can be misleading in imbalanced datasets, which do not represent the actual model performance. Therefore, the usual suspects of performance measurement in machine learning: Precision, Recall, and F-Measure, are more commonly reported, mainly in detection tasks (e.g., [49,168,178]. Those measurements are used to solve the problem of accuracy for imbalanced datasets. However, their existence is not omnipotent, and each of the performance measurements has its own intent and usage. Especially, a general measurement of F1-score cannot represent the performance for all tasks. For example, for the vulnerability detection task, recall has more importance than precision because it is better to falsely predict a good source code as vulnerable rather than a vulnerable source code as good. This is shown in [77], where they only use Accuracy and F1-score, where recall should be put in the spotlight, like in [217]. That example reflects that some studies use staple performance measurement merely as a formality, without aligning it with the actual needs of the problem that the model is intended to solve.

New problems may need different metrics, hence newer performance measurements are being developed as well. Area Under the Curve (AUC) is used to provide a threshold-independent measure of classifier performance, especially in multi-class detection tasks (e.g., [132,181]). Various mean-based metrics commonly appear in search and recommendation settings such as Mean Reciprocal Rank (MRR), Mean Statement Recall (MSR), Mean Statement Precision (MSP) and Mean Average Precision (MAP) (e.g., [174,195,215]. A newer version of that performance adds a modification like Top-K (e.g., [150,176], and it is used mainly for search tasks because the task may produce multiple results. Lastly, newest metrics such as BLEU, METEOR and ROUGE are used specifically for the newer and more complicated generation and summarization tasks (e.g., [244,245]). For typical and common machine learning tasks such as detection, ubiquitous methods are good to go. But for newer tasks such as summarization and generation, new methods need to be used to keep the relevance going. At the end, choosing which measurement is not less important than choosing models in the artificial intelligence research world.

5.4. RQ4: Gaps, challenges and opportunities

While this study has revealed clear methodological trends and progress in the application of AI to source code understanding, it has also uncovered meaningful gaps and underexplored areas across tasks, techniques and evaluation practices. These limitations offer both cautionary insights and promising directions for future research and practice. The following subsections outline these issues from two perspectives: first, the implications for researchers aiming to advance the scientific frontier; and second, the practical considerations relevant to software engineering practitioners who may adopt or build upon these technologies.

5.4.1. Challenges and opportunities for researchers

In the detection task, while competition on performance is still ongoing, a different aspect of detection should start to be in focus. The competition of getting better performance, even in just 1%, is possible because of the abundant datasets (more than 4 datasets that have been used in more than 15 studies, as discussed in Section 5.1), well-structured problem formulations and established metrics. As a result, it is often the default target when applying new models. However, the field now faces diminishing novelty. Usage of convolutional-based network was used since 2019 [125], 2020 [170], until 2023 [55]. Even newer ones like CodeBERT have been used multiple times in the last 2 years (2023 [88,145,202] and 2024 [44,138,239]). Therefore, in the future, researchers should aim beyond incremental improvements, focusing instead on inventing new methods or hybrid architectures. Importantly, few detection studies incorporate explainability, only 6 out of 168 (3.57%) (e.g., [69,74]). As detection systems are increasingly deployed in security and safety-critical settings, a new focus on making model predictions interpretable is both necessary and overdue.

Search & understanding tasks need a push towards innovation. The search task has started to stagnate, making it necessary to take a new approach in the future. Most retrieval frameworks now converge around dual encoders or transformer-based (37.09% or 23 out of 62) and little architectural innovation has occurred in recent years (7 out of 9 in 2024 are using transformer-based models, e.g., [205–207]). By contrast, understanding task remains a broad (all 4 studies [213,237–239] in 2024 all have different tasks) and evolving domain. Recent studies tend to target emerging subtopics, such as detecting comment scope or design pattern detection, yet few propose competing methodological frameworks. At the same time, understanding tasks sits at the heart of explainability [222]: they involve aligning code semantics with human goals. Exploring understanding possibilities across topics may direct near-future work direction into areas such as interpretable embeddings, context-aware reasoning and user-aligned evaluation. Now is the moment to define impactful research in the comprehension of source code. Therefore, creating new implementations, new techniques and new problems in these tasks will be the next path.

Among all task groups, generation and summarization remain the least developed and maybe will still be. The research base is small, as seen in Section 4.2.3 and task definitions vary widely. Consequently, no clear methodological trend has yet emerged. However, the rise of large language models (LLMs) such as CodeT5 [254] and StarCoder [239] has posed a disruption. Researchers must now define what a “good” generated code/words looks like, moving beyond BLEU and ROUGE toward human-centered evaluation, consistency and factuality. However, this task is no less important than the other because generation will help in the post-processing for explainability (e.g., uncover and explain implicit knowledge in code [222]). This task is another promising and

challenging direction that remains largely untapped by brave computer scientists.

Across the task, performance measurement usage is still underdiscussed. As discussed in Section 5.3 subsection performance measurement, a lot of variations of performance measurement are used, but not all are explained. Some, especially the ubiquitous ones, are used just as a formality, as a means of research and comparability with other research. They do not explain the reasons why they chose that specific set of performance measurements. Thankfully, some studies really used the performance as their intended use to measure the model's performance based on a certain task, and they explained the reason. Having an explanation of the performance measurement used should have been the standard for all research in this AI area.

In summary, open challenges for researchers are around the explainability of the model. In detail, the challenges for each task are:

1. **Explainability:** Model interpretability for detection systems deployed in security and safety-critical settings.
2. **Comprehension:** A searching algorithm that understands the source code for more understandable search results.
3. **Explanation Generation:** Post-processing of explainability by the model to increase human understanding.
4. **Performance Measurement:** Explained and standardized performance measurement for comparability across tasks.

5.4.2. Challenges and opportunities for practitioners

Most practitioners are looking for a stable, producible and fast delivery of AI tools. Among the tasks discussed in this study, detection tools are the most deployable. Even in 2022 [118], there has been talk about the deployment of vulnerability detection. Furthermore, they are backed by large datasets, robust models and task-aligned metrics. Many detection approaches already support multiple programming languages, as shown in Fig. 3(b) and therefore could be integrated into continuous integration (CI) pipelines with minimal risk. In search tasks, little has changed from a practitioner's perspective. Existing models are functional and reliable, though not substantially better than traditional lexical search in all cases [199] in 2018 and [196] in 2019). In contrast, code understanding tools represent a compelling but fragile frontier. These systems can assist with onboarding, maintenance and traceability, but they must balance interpretability with performance and stability. As proof, only 9 out of 20 studies have accuracy more than 90% (e.g., [175,231]. For now, they are better suited to exploratory tooling than production deployment. Lastly, generative models are not yet stable enough for widespread use in safety-critical or high-assurance systems, with only one study that has a BLEU score a little more than 48 [240] (Highest BLEU score is 100). However, for forward-leaning teams and tool builders, this is a frontier area worth exploring. As LLMs continue to improve, startups and large platforms alike may gain a competitive advantage by pioneering human-aligned generation pipelines. The open question is whether this space will remain accessible or be dominated by resource-rich companies with proprietary models.

In summary, open challenges for practitioners are around the stability of the model. In detail, the challenges for each task are:

1. **Deployability:** Ensuring trustworthy, reproducible, and efficient deployment of AI models in real-world software systems.
2. **Multiple Programming Languages:** Maintaining consistent accuracy and generalization across languages with different syntax and semantics.
3. **Stability vs. Interpretability:** Balancing interpretability with performance and stability, especially for code understanding tools used in critical tasks.
4. **Accessibility:** It remains an open question whether advancements in LLM-based code generation will be democratized for the wider community or dominated by resource-rich companies with proprietary models, limiting open research and adoption.

Summary of RQ 4 findings: Explainability remains a critical yet overlooked challenge across the AI-for-code pipeline. Despite its importance to both researchers and practitioners, few studies provide transparent reasoning or interpretable outputs, limiting trust, adoption and impact.

6. Threat to validity

This study followed systematic mapping guidelines, prioritizing a general view rather than a deep technique discussion, yet several validity concerns remain. Construct validity may be affected by inconsistent terminology across primary studies; to mitigate this, a task-based taxonomy and dual-review process were used. Internal validity risks include subjective decisions in classification and manual data extraction, though consistent inclusion criteria and iterative checks were applied. External validity is limited by the exclusion of grey literature, preprints, and industry reports, which may overlook recent or non-peer-reviewed advances. While this preserves peer-reviewed quality standards, it may result in the omission of cutting-edge or emerging research, particularly relevant given the rapid evolution of machine learning models. Additionally, limiting the study to English-language publications potentially underrepresents contributions from non-English research communities, introducing a language bias that may narrow the global applicability of the findings. Finally, reliability is supported by standardized procedures and shared materials, though full replication may still involve interpretive judgment. Rapid changes in AI techniques in NLP can make this study obsolete at a faster pace, but future work can strengthen validity through broader reviewer participation, automation support, and more inclusive modern data sources.

7. Conclusion

This systematic mapping study investigated how artificial intelligence techniques, particularly natural language processing and machine learning, have been applied to source code understanding. Based on an analysis of over 220 peer-reviewed studies, we identified dominant task categories (detection, search & understanding, generation & summarization), characterized trends in preprocessing strategies and code embedding, and reviewed the evolution of modeling approaches. Deep learning, especially transformer-based and graph-based models, has emerged as the preferred modeling paradigm, while explainability and evaluation practices remain underdeveloped.

Despite growing maturity in detection and search tasks, substantial gaps persist in explainability, model transparency and standardized evaluation, especially for generative applications. These challenges present valuable opportunities for future research. By highlighting methodological patterns, task-level insights and critical gaps, this study offers a comprehensive reference for researchers and practitioners seeking to advance intelligent software engineering systems.

CRediT authorship contribution statement

Dzikri Rahadian Fudholi: Writing – review & editing, Writing – original draft, Visualization, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Andrea Capiluppi:** Writing – review & editing, Validation, Supervision, Project administration, Investigation.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Dzikri Rahadian Fudholi reports financial support was provided by Indonesia Endowment Fund for Education. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by the Indonesia Endowment Fund for Education (Lembaga Pengelola Dana Pendidikan – LPDP), Ministry of Finance of the Republic of Indonesia, under the doctoral scholarship program. The authors gratefully acknowledge LPDP's funding and continued support, which made this work possible.

Data availability

The dataset supporting the findings of this study is openly available on Zenodo at https://zenodo.org/records/16947597 [32].

References

[1] T.R. Tulili, A. Capiluppi, A. Rastogi, Burnout in software engineering: A systematic mapping study, Inf. Softw. Technol. 155 (2023) 107116.
[2] R.W. Sebesta, Concepts of Programming Languages, tenth ed., Pearson Education, 2012.
[3] Z. Wang, L. Zhang, C. Cao, N. Luo, X. Luo, P. Liu, How does naming affect language models on code analysis tasks? J. Softw. Eng. Appl. 17 (2024) 803–816, http://dx.doi.org/10.4236/jsea.2024.1711044.
[4] M. Allamanis, E.T. Barr, C. Bird, C. Sutton, A survey of machine learning for big code and naturalness, ACM Comput. Surv. 51 (4) (2018) 1–37.
[5] B. Vasilescu, V. Filkov, A. Serebrenik, Gender and tenure diversity in GitHub teams, in: Proceedings of the 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, IEEE, 2015, pp. 321–331.
[6] B. Ray, V.J. Hellendoorn, S. Godhane, Z. Tu, A. Bacchelli, P. Devanbu, The naturalness of software, in: Proceedings of the 38th International Conference on Software Engineering, ACM, 2016, pp. 837–848.
[7] B. Kitchenham, O.P. Brereton, D. Budgen, M. Turner, J. Bailey, S. Linkman, Systematic literature reviews in software engineering–a systematic literature review, Inf. Softw. Technol. 51 (1) (2009) 7–15.
[8] K. Petersen, S. Vakkalanka, L. Kuzniarz, Guidelines for conducting systematic mapping studies in software engineering: An update, Inf. Softw. Technol. 64 (2015) 1–18.
[9] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, Inf. Softw. Technol. 58 (2015) 92–107.
[10] N. Shiri Harzevili, A. Boaye Belle, J. Wang, S. Wang, Z.M. Jiang, N. Nagappan, A systematic literature review on automated software vulnerability detection using machine learning, ACM Comput. Surv. 57 (3) (2024) 1–36.
[11] S. Pooja, C. Chandrakala, K.R. Laiju, Developer's roadmap to design software vulnerability detection model using different ai approaches, IEEE Access 10 (2022) 75637–75656.
[12] M. Mehrabi Koushki, I. AbuAlhaol, A.D. Raju, Y. Zhou, R.S. Giagone, H. Shengqiang, On building machine learning pipelines for android malware detection: a procedural survey of practices, challenges and opportunities, Cybersecurity 5 (1) (2022) 16.
[13] A. Joomye, M.H. Ling, K.-L.A. Yau, A brief survey of deep learning methods for android malware detection, Int. J. Syst. Assur. Eng. Manag. (2024) 1–23.
[14] T. Bilot, N. El Madhoun, K. Al Agha, A. Zouaoui, A survey on malware detection with graph representation learning, ACM Comput. Surv. 56 (11) (2024) 1–36.
[15] L. Zhao, A study of code search technology, in: 2024 7th International Conference on Computer Information Science and Application Technology, CISAT, IEEE, 2024, pp. 603–608.
[16] Y. Xie, J. Lin, H. Dong, L. Zhang, Z. Wu, Survey of code search based on deep learning, ACM Trans. Softw. Eng. Methodol. 33 (2) (2023) 1–42.
[17] Z. Li, Y. Song, P. Chen, D. Li, G. Gong, K. Lv, Progress of word embedding in code generation, in: 2019 IEEE 19th International Conference on Software Quality, Reliability and Security Companion (QRS-C), IEEE, 2019, pp. 538–539.
[18] T. Martinet, G. Cleuziou, M. Exbrayat, F. Flouvat, From document to program embeddings: can distributional hypothesis really be used on programming languages? in: ECAI 2024, IOS Press, 2024, pp. 2138–2145.
[19] M. Hadj-Kacem, N. Bouassida, Application of deep learning for code smell detection: Challenges and opportunities, SN Comput. Sci. 5 (5) (2024) 614.
[20] H. Zhang, K. Zhang, Z. Li, J. Li, J. Li, Y. Li, Y. Zhao, Y. Zhu, F. Liu, G. Li, et al., Deep learning for code generation: a survey, Sci. China Inf. Sci. 67 (9) (2024) 191101.
[21] H. Wang, B. Cui, Q. Yuan, R. Shi, M. Huang, A review of deep learning based malware detection techniques, Neurocomputing (2024) 128010.
[22] A. Karmakar, R. Robbes, What do pre-trained code models know about code? in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2021, pp. 1332–1336.
[23] L. Aversano, M.L. Bernardi, M. Cimitile, M. Iammarino, D. Montano, Forecasting technical debt evolution in software systems: an empirical study, Front. Comput. Sci. 17 (3) (2023) 173210.
[24] J. Wang, G. Ta, F. Meng, Development trend of code defect detection technology based on natural language processing, in: 2024 IEEE 13th International Conference on Communication Systems and Network Technologies, CSNT, IEEE, 2024, pp. 459–463.
[25] M. Lei, H. Li, J. Li, N. Aundhkar, D.-K. Kim, Deep learning application on code clone detection: A review of current knowledge, J. Syst. Softw. 184 (2022) 111141.
[26] X. Zhang, X. Hou, X. Qiao, W. Song, A review of automatic source code summarization, Empir. Softw. Eng. 29 (6) (2024) 162.
[27] G.S. Sodhi, D. Rattan, An insight on software features supporting software transplantation: a systematic review, Arch. Comput. Methods Eng. 29 (1) (2022) 275–312.
[28] H.P. Samoaa, F. Bayram, P. Salza, P. Leitner, A systematic mapping study of source code representation for deep learning in software engineering, IET Softw. 16 (4) (2022) 351–385.
[29] C. Niu, C. Li, V. Ng, D. Chen, J. Ge, B. Luo, An empirical comparison of pre-trained models of source code, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE, 2023, pp. 2136–2148.
[30] Y. Wan, Z. Bi, Y. He, J. Zhang, H. Zhang, Y. Sui, G. Xu, H. Jin, P. Yu, Deep learning for code intelligence: Survey, benchmark and toolkit, ACM Comput. Surv. 56 (12) (2024) 1–41.
[31] Q. Zhang, C. Fang, Y. Ma, W. Sun, Z. Chen, A survey of learning-based automated program repair, ACM Trans. Softw. Eng. Methodol. 33 (2) (2023) 1–69.
[32] D.R. Fudholi, A. Capiluppi, Studies dataset for "artificial intelligence for source code understanding tasks: A systematic mapping study", Zenodo, 2025. http://dx.doi.org/10.5281/zenodo.16947597, https://zenodo.org/records/16947597.
[33] A. Avetisyan, S. Kurmangaleev, S. Sargsyan, M. Arutunian, A. Belevantsev, LLVM-based code clone detection framework, in: 2015 Computer Science and Information Technologies, CSIT, IEEE, 2015, pp. 100–104.
[34] H. Liu, S. Jiang, X. Qi, Y. Qu, H. Li, T. Li, C. Guo, S. Guo, Detect software vulnerabilities with weight biases via graph neural networks, Expert Syst. Appl. 238 (2024) 121764.
[35] A. Bahaa, A.E.-R. Kamal, H. Fahmy, A.S. Ghoneim, DB-CBIL: A DistilBert-based transformer hybrid model using CNN and BiLSTM for software vulnerability detection, IEEE Access (2024).
[36] Z. Cai, Y. Cai, X. Chen, G. Lu, W. Pei, J. Zhao, CSVD-TF: Cross-project software vulnerability detection with TrAdaBoost by fusing expert metrics and semantic metrics, J. Syst. Softw. 213 (2024) 112038.
[37] X. Li, Y. Lin, Y. Zheng, J. He, R. Liao, J. Wang, BABEL: A novel software vulnerability detection framework for breaking language barriers, in: 2024 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2024, pp. 599–611.
[38] J. Wu, X. Ling, X. Duan, T. Luo, M. Yang, Vuldl: Tree-based and graph-based neural networks for vulnerability detection and localization, in: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, 2024, pp. 323–332.
[39] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, H. Jin, Vulcnn: An image-inspired scalable vulnerability detection system, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2365–2376.
[40] X. Zhang, F. Zhang, B. Zhao, B. Zhou, B. Xiao, Vuld-transformer: source code vulnerability detection via transformer, in: Proceedings of the 14th Asia-Pacific Symposium on Internetware, 2023, pp. 185–193.
[41] X. Du, S. Zhang, Y. Zhou, H. Du, A vulnerability severity prediction method based on bimodal data and multi-task learning, J. Syst. Softw. 213 (2024) 112039.
[42] P. Feng, L. Yang, D. Lu, N. Xi, J. Ma, Bejagnn: behavior-based java malware detection via graph neural network, J. Supercomput. 79 (14) (2023) 15390–15414.
[43] A. Narayanan, M. Chandramohan, L. Chen, Y. Liu, A multi-view context-aware approach to android malware detection and malicious code localization, Empir. Softw. Eng. 23 (2018) 1222–1274.
[44] H. Bao, L. Wei, Multi-label code smell detection method based on codebert, in: 2024 3rd International Conference on Electronics and Information Technology, EIT, IEEE, 2024, pp. 1–7.
[45] A. Kovačević, J. Slivka, D. Vidaković, K.-G. Grujić, N. Luburić, S. Prokić, G. Sladić, Automatic detection of long method and god class code smells through neural source code embeddings, Expert Syst. Appl. 204 (2022) 117607.

[46] S. Sahar, M. Younas, M.M. Khan, M.U. Sarwar, DP-CCL: A supervised contrastive learning approach using codebert model in software defect prediction, IEEE Access 12 (2024) 22582–22594.

[47] W. Zhuang, H. Wang, X. Zhang, Just-in-time defect prediction based on AST change embedding, Knowl.-Based Syst. 248 (2022) 108852.

[48] W. Yao, M. Shafiq, X. Lin, X. Yu, A software defect prediction method based on program semantic feature mining, Electronics 12 (7) (2023) 1546.

[49] J. Wang, M. Xu, H. Chen, Detecting source code vulnerabilities using high-precision code representation and bimodal contrastive learning, in: 2024 International Conference on Networking and Network Applications (NaNA), IEEE, 2024, pp. 536–541.

[50] G. Zhang, Z. Jin, T. Yao, J. Qin, ECET: Enhanced vulnerability detection through code embedding techniques, in: 2024 Sixth International Conference on Next Generation Data-Driven Networks, NGDN, IEEE, 2024, pp. 100–103.

[51] M.S. Akter, H. Shahriar, J.R. Cardenas, S.I. Ahamed, A. Cuzzocrea, Feature engineering-based detection of buffer overflow vulnerability in source code using neural networks, in: 2023 IEEE 47th Annual Computers, Software, and Applications Conference, COMPSAC, IEEE, 2023, pp. 765–776.

[52] Y. Chen, Z. Lin, Z. Guo, Application of hierarchical attention network in vulnerability detection model, in: 2024 2nd International Conference on Big Data and Privacy Computing, BDPC, IEEE, 2024, pp. 43–48.

[53] Z. Liu, Z. Tang, J. Zhang, X. Xia, X. Yang, Pre-training by predicting program dependencies for vulnerability analysis tasks, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13.

[54] Y. Chenxi, W. Le, CSCVD: A method of source code vulnerability detection based on semantic feature and context dependence, in: 2023 9th International Conference on Computer and Communications, ICCC, IEEE, 2023, pp. 2162–2169.

[55] R.S. Mim, A. Khatun, T. Ahammed, K. Sakib, Impact of centrality on automated vulnerability detection using convolutional neural network, in: 2023 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD), IEEE, 2023, pp. 331–335.

[56] J. Zhou, M. Pacheco, Z. Wan, X. Xia, D. Lo, Y. Wang, A.E. Hassan, Finding a needle in a haystack: Automated mining of silent vulnerability fixes, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2021, pp. 705–716.

[57] X. Cheng, H. Wang, J. Hua, M. Zhang, G. Xu, L. Yi, Y. Sui, Static detection of control-flow-related vulnerabilities using graph embedding, in: 2019 24th International Conference on Engineering of Complex Computer Systems, ICECCS, IEEE, 2019, pp. 41–50.

[58] A. Diwan, M.Q. Li, B.C. Fung, Vdgraph2vec: Vulnerability detection in assembly code using message passing neural networks, in: 2022 21st IEEE International Conference on Machine Learning and Applications, ICMLA, IEEE, 2022, pp. 1039–1046.

[59] B. Steenhoek, H. Gao, W. Le, Dataflow analysis-inspired deep learning for efficient vulnerability detection, in: Proceedings of the 46th Ieee/Acm International Conference on Software Engineering, 2024, pp. 1–13.

[60] Y. Yu, S. Gan, X. Qin, Firm vulseeker: Bert and siamese based vulnerability for embedded device firmware images, in: 2021 IEEE Symposium on Computers and Communications, ISCC, IEEE, 2021, pp. 1–7.

[61] X. Cheng, G. Zhang, H. Wang, Y. Sui, Path-sensitive code embedding via contrastive learning for software vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 519–531.

[62] W. Ma, M. Zhao, E. Soremekun, Q. Hu, J.M. Zhang, M. Papadakis, M. Cordy, X. Xie, Y.L. Traon, Graphcode2vec: Generic code embedding via lexical and program dependence analyses, in: Proceedings of the 19th International Conference on Mining Software Repositories, 2022, pp. 524–536.

[63] S. Liu, W. Ma, J. Wang, X. Xie, R. Feng, Y. Liu, Enhancing code vulnerability detection via vulnerability-preserving data augmentation, in: Proceedings of the 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems, 2024, pp. 166–177.

[64] H. Wang, Z. Tang, S.H. Tan, J. Wang, Y. Liu, H. Fang, C. Xia, Z. Wang, Combining structured static code information and dynamic symbolic traces for software vulnerability prediction, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13.

[65] B. Yuan, Y. Lu, Y. Fang, Y. Wu, D. Zou, Z. Li, Z. Li, H. Jin, Enhancing deep learning-based vulnerability detection by building behavior graph model, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE, 2023, pp. 2262–2274.

[66] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, X. Wu, C. Tao, T. Zhang, W. Liu, Learning to detect memory-related vulnerabilities, ACM Trans. Softw. Eng. Methodol. 33 (2) (2023) 1–35.

[67] S. Cao, X. Sun, L. Bo, Y. Wei, B. Li, Bgnn4vd: Constructing bidirectional graph neural-network for vulnerability detection, Inf. Softw. Technol. 136 (2021) 106576.

[68] Z. Song, J. Xu, K. Li, Z. Shan, HCRVD: A vulnerability detection system based on CST-pdg hierarchical code representation learning., Comput. Mater. Contin. 79 (3) (2024).

[69] C. Liu, X. Chen, X. Li, Y. Xue, Making vulnerability prediction more practical: Prediction, categorization, and localization, Inf. Softw. Technol. 171 (2024) 107458.

[70] S.M. Ghaffarian, H.R. Shahriari, Neural software vulnerability analysis using rich intermediate graph representations of programs, Inform. Sci. 553 (2021) 189–207.

[71] C. Zhang, Y. Xin, Static vulnerability detection based on class separation, J. Syst. Softw. 206 (2023) 111832.

[72] R. Xu, Z. Tang, G. Ye, H. Wang, X. Ke, D. Fang, Z. Wang, Detecting code vulnerabilities by learning from large-scale open source repositories, J. Inf. Secur. Appl. 69 (2022) 103293.

[73] R. Wang, S. Xu, Y. Tian, X. Ji, X. Sun, S. Jiang, SCL-cvd: Supervised contrastive learning for code vulnerability detection via GraphCodeBERT, Comput. Secur. 145 (2024) 103994.

[74] X. Yuan, G. Lin, H. Mei, Y. Tai, J. Zhang, Software vulnerable functions discovery based on code composite feature, J. Inf. Secur. Appl. 81 (2024) 103718.

[75] T. Ganz, E. Imgrund, M. H¨arterich, K. Rieck, PAVUDI: Patch-based vulnerability discovery using machine learning, in: Proceedings of the 39th Annual Computer Security Applications Conference, 2023, pp. 704–717.

[76] C. McLaughlin, Y. Lu, Multi-class vulnerability prediction using value flow and graph neural networks, Neural Comput. Appl. 36 (25) (2024) 15869–15891.

[77] B. Wu, S. Liu, Y. Xiao, Z. Li, J. Sun, S.-W. Lin, Learning program semantics for vulnerability detection via vulnerability-specific inter-procedural slicing, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023, pp. 1371–1383.

[78] Z. Ren, X. Ju, X. Chen, H. Shen, ProRLearn: boosting prompt tuning-based vulnerability detection by reinforcement learning, Autom. Softw. Eng. 31 (2) (2024) 38.

[79] S. Zhao, J. Zhu, J. Peng, Software vulnerability mining and analysis based on deep learning., Comput. Mater. Contin. 80 (2) (2024).

[80] Z. Tian, B. Tian, J. Lv, Y. Chen, L. Chen, Enhancing vulnerability detection via AST decomposition and neural sub-tree encoding, Expert Syst. Appl. 238 (2024) 121865.

[81] C. Ni, X. Yin, K. Yang, D. Zhao, Z. Xing, X. Xia, Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023, pp. 1611–1622.

[82] G. Lu, X. Ju, X. Chen, W. Pei, Z. Cai, GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning, J. Syst. Softw. 212 (2024) 112031.

[83] J. Yang, O. Ruan, J. Zhang, Tensor-based gated graph neural network for automatic vulnerability detection in source code, Softw. Test. Verif. Reliab. 34 (2) (2024) e1867.

[84] W. Xiao, Z. Hou, T. Wang, C. Zhou, C. Pan, Msgvul: Multi-semantic integration vulnerability detection based on relational graph convolutional neural networks, Inf. Softw. Technol. (2024) 107442.

[85] W. Cai, J. Chen, J. Yu, L. Gao, A software vulnerability detection method based on deep learning with complex network analysis and subgraph partition, Inf. Softw. Technol. 164 (2023) 107328.

[86] S. Nguyen, T.-T. Nguyen, T.T. Vu, T.-D. Do, K.-T. Ngo, H.D. Vo, Code-centric learning-based just-in-time vulnerability detection, J. Syst. Softw. 214 (2024) 112014.

[87] Z. Chu, Y. Wan, Q. Li, Y. Wu, H. Zhang, Y. Sui, G. Xu, H. Jin, Graph neural networks for vulnerability detection: A counterfactual explanation, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024, pp. 389–401.

[88] H.D. Vo, S. Nguyen, Can an old fashioned feature extraction and a light-weight model improve vulnerability type identification performance? Inf. Softw. Technol. 164 (2023) 107304.

[89] T. Sun, W. Pian, N. Daoudi, K. Allix, T. F. Bissyandé, J. Klein, Laficmil: Rethinking large file classification from the perspective of correlated multiple instance learning, in: International Conference on Applications of Natural Language To Information Systems, Springer, 2024, pp. 62–77.

[90] M.F. Rozi, T. Ban, S. Ozawa, A. Yamada, T. Takahashi, D. Inoue, Securing code with context: Enhancing vulnerability detection through contextualized graph representations, IEEE Access (2024).

[91] J. Wang, H. Xiao, S. Zhong, Y. Xiao, DeepVulSeeker: A novel vulnerability identification framework via code graph structure and pre-training mechanism, Future Gener. Comput. Syst. 148 (2023) 15–26.

[92] S. Shimmi, A. Rahman, M. Gadde, H. Okhravi, M. Rahimi, {VulSim}: Leveraging similarity of {multi-dimensional} neighbor embeddings for vulnerability detection, in: 33rd USENIX Security Symposium (USENIX Security 24), 2024, pp. 1777–1794.

[93] Q. Zeng, D. Xiong, Z. Wu, K. Qian, Y. Wang, Y. Su, Tacsan: Enhancing vulnerability detection with graph neural network, Electronics 13 (19) (2024) 3813.

[94] R. Fuladi, K. Hanifi, CodeGrapher: An image representation method to enhance software vulnerability prediction., in: ENASE, 2024, pp. 666–673.

[95] S. Pujar, Y. Zheng, L. Buratti, B. Lewis, Y. Chen, J. Laredo, A. Morari, E. Epstein, T. Lin, B. Yang, et al., Analyzing source code vulnerabilities in the D2A dataset with ml ensembles and C-BERT, Empir. Softw. Eng. 29 (2) (2024) 48.
[96] X. Cheng, X. Nie, N. Li, H. Wang, Z. Zheng, Y. Sui, How about bug-triggering paths?-understanding and characterizing learning-based vulnerability detectors, IEEE Trans. Dependable Secur. Comput. 21 (2) (2022) 542–558.
[97] Y. Zhao, L. Gong, Z. Huang, Y. Wang, M. Wei, F. Wu, Coding-ptms: How to find optimal code pre-trained models for code embedding in vulnerability detection? in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1732–1744.
[98] H. Sun, L. Cui, L. Li, Z. Ding, S. Li, Z. Hao, H. Zhu, Vdtriplet: Vulnerability detection with graph semantics using triplet model, Comput. Secur. 139 (2024) 103732.
[99] P. Xiao, Q. Xiao, X. Zhang, Y. Wu, F. Yang, Vulnerability detection based on enhanced graph representation learning, IEEE Trans. Inf. Forensics Secur. (2024).
[100] M. Hoq, S.R. Chilla, M. Ahmadi Ranjbar, P. Brusilovsky, B. Akram, Sann: Programming code representation using attention neural network with optimized subtree extraction, in: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, 2023, pp. 783–792.
[101] T. Yang, S. Lian, Q. Jia, C. Hu, S. Guo, Multi-granularity deep vulnerability detection using graph neural networks, in: International Conference on Neural Information Processing, Springer, 2023, pp. 152–164.
[102] J. Zhang, Z. Liu, X. Hu, X. Xia, S. Li, Vulnerability detection by learning from syntax-based execution paths of code, IEEE Trans. Softw. Eng. 49 (8) (2023) 4196–4212.
[103] W. Tang, M. Tang, M. Ban, Z. Zhao, M. Feng, CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection, J. Syst. Softw. 199 (2023) 111623.
[104] D. Hin, A. Kan, H. Chen, M.A. Babar, Linevd: Statement-level vulnerability detection using graph neural networks, in: Proceedings of the 19th International Conference on Mining Software Repositories, 2022, pp. 596–607.
[105] H. Wang, G. Ye, Z. Tang, S.H. Tan, S. Huang, D. Fang, Y. Feng, L. Bian, Z. Wang, Combining graph-based learning with automated data collection for code vulnerability detection, IEEE Trans. Inf. Forensics Secur. 16 (2020) 1943–1958.
[106] X.-C. Wen, Y. Chen, C. Gao, H. Zhang, J.M. Zhang, Q. Liao, Vulnerability detection with graph simplification and enhanced graph representation learning, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE, 2023, pp. 2275–2286.
[107] X. Cheng, Vulnerability detection via typestate-guided code representation learning, in: International Conference on Formal Engineering Methods, Springer, 2023, pp. 291–297.
[108] J. Gear, Y. Xu, E. Foo, P. Gauravaram, Z. Jadidi, L. Simpson, SCEVD: Semantic-enhanced code embedding for vulnerability discovery, in: 2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), IEEE, 2022, pp. 1522–1527.
[109] C. Ni, X. Guo, Y. Zhu, X. Xu, X. Yang, Function-level vulnerability detection through fusing multi-modal knowledge, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2023, pp. 1911–1918.
[110] X. Xia, Y. Wang, Y. Yang, Source code vulnerability detection based on SARgin, in: 2021 2nd International Conference on Electronics, Communications and Information Technology, CECIT, IEEE, 2021, pp. 1144–1149.
[111] C. Ni, L. Shen, W. Wang, X. Chen, X. Yin, L. Zhang, FVA: Assessing function-level vulnerability by integrating flow-sensitive structure and code statement semantic, in: 2023 IEEE/ACM 31st International Conference on Program Comprehension, ICPC, IEEE, 2023, pp. 339–350.
[112] A. Fidalgo, I. Medeiros, P. Antunes, N. Neves, Towards a deep learning model for vulnerability detection on web application variants, in: 2020 IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW, IEEE, 2020, pp. 465–476.
[113] W. Yu, G. Kou, Q. Liu, H. Zhang, Q. Wang, PATVD: Vulnerability detection based on pre-training techniques and adversarial training, in: 2022 IEEE Smartworld, Ubiquitous Intelligence & Computing, Scalable Computing & Communications, Digital Twin, Privacy Computing, Metaverse, Autonomous & Trusted Vehicles (SmartWorld/UIC/ScalCom/DigitalTwin/PriComp/Meta), IEEE, 2022, pp. 1774–1781.
[114] H.V. Nguyen, J. Zheng, A. Inomata, T. Uehara, Code aggregate graph: Effective representation for graph neural networks to detect vulnerable code, IEEE Access 10 (2022) 123786–123800.
[115] D. Cao, J. Huang, X. Zhang, X. Liu, Ftclnet: Convolutional LSTM with Fourier transform for vulnerability detection, in: 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), IEEE, 2020, pp. 539–546.
[116] J. Gear, Y. Xu, E. Foo, P. Gauravaram, Z. Jadidi, L. Simpson, Software vulnerability detection using informed code graph pruning, IEEE Access 11 (2023) 135626–135644.
[117] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, Deepwukong: Statically detecting software vulnerabilities using deep graph neural network, ACM Trans. Softw. Eng. Methodol. (TOSEM) 30 (3) (2021) 1–33.
[118] G. Lin, H. Jia, D. Wu, Distilled and contextualized neural models benchmarked for vulnerable function detection, Mathematics 10 (23) (2022) 4482.
[119] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, Adv. Neural Inf. Process. Syst. 32 (2019).
[120] J. Liu, X. Zhu, C. Liu, X. Cui, Q. Liu, CPGBERT: An effective model for defect detection by learning program semantics via code property graph, in: 2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), IEEE, 2022, pp. 274–282.
[121] J. Yang, C. Fu, F. Deng, M. Wen, X. Guo, C. Wan, Toward interpretable graph tensor convolution neural network for code semantics embedding, ACM Trans. Softw. Eng. Methodol. 32 (5) (2023) 1–40.
[122] P. Keller, A.K. Kaboré, L. Plein, J. Klein, Y. Le Traon, T.F. Bissyandé, What you see is what it means! semantic representation learning of code based on visualization and transfer learning, ACM Trans. Softw. Eng. Methodol. (TOSEM) 31 (2) (2021) 1–34.
[123] M. Fan, A. Jia, J. Liu, T. Liu, W. Chen, When representation learning meets software analysis, in: Proceedings of the 1st ACM SIGSOFT International Workshop on Representation Learning for Software Engineering and Program Languages, 2020, pp. 17–18.
[124] H.K. Dam, T. Pham, S.W. Ng, T. Tran, J. Grundy, A. Ghose, T. Kim, C.-J. Kim, Lessons learned from using a deep tree-based model for software defect prediction in practice, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories, MSR, IEEE, 2019, pp. 46–57.
[125] Z. Cai, L. Lu, S. Qiu, An abstract syntax tree encoding method for cross-project defect prediction, IEEE Access 7 (2019) 170844–170853.
[126] N. Luo, Y. Ma, J. Li, Y. Xie, A just-in-time software defect detection method using generative adversarial networks, in: 2023 4th International Conference on Electronic Communication and Artificial Intelligence, ICECAI, IEEE, 2023, pp. 37–45.
[127] W. Zheng, L. Tan, C. Liu, Software defect prediction method based on transformer model, in: 2021 IEEE International Conference on Artificial Intelligence and Computer Applications, ICAICA, IEEE, 2021, pp. 670–674.
[128] H. Yang, W. Le, L. Yumin, A defect detection method based on code defect knowledge graph, in: 2023 9th International Conference on Computer and Communications, ICCC, IEEE, 2023, pp. 2104–2112.
[129] J. Chang, Z. Ma, B. Cao, E. Zhu, Vdda: An effective software vulnerability detection model based on deep learning and attention mechanism, in: 2023 26th International Conference on Computer Supported Cooperative Work in Design, CSCWD, IEEE, 2023, pp. 474–479.
[130] J. Deng, L. Lu, S. Qiu, Y. Ou, A suitable AST node granularity and multi-kernel transfer convolutional neural network for cross-project defect prediction, IEEE Access 8 (2020) 66647–66661.
[131] J. Huang, J. Nie, Y. Gong, W. You, B. Liang, P. Bian, Raisin: Identifying rare sensitive functions for bug detection, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–12.
[132] Y. Li, S. Wang, T.N. Nguyen, Contextuality of code representation learning, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2023, pp. 548–559.
[133] W. Liu, Y. Yue, X. Chen, Q. Gu, P. Zhao, X. Liu, J. Zhao, Sedpgk: Semi-supervised software defect prediction with graph representation learning and knowledge distillation, Inf. Softw. Technol. 174 (2024) 107510.
[134] J. Liu, J. Ai, M. Lu, J. Wang, H. Shi, Semantic feature learning for software defect prediction from source code and external knowledge, J. Syst. Softw. 204 (2023) 111753.
[135] Y. Xing, X. Qian, Y. Guan, B. Yang, Y. Zhang, Cross-project defect prediction based on G-LSTM model, Pattern Recognit. Lett. 160 (2022) 50–57.
[136] Y. Chen, C. Xu, J.S. He, S. Xiao, F. Shen, Compiler IR-based program encoding method for software defect prediction., Comput. Mater. Contin. 72 (3) (2022).
[137] Y. Fang, C. Huang, M. Zeng, Z. Zhao, C. Huang, Jstrong: Malicious JavaScript detection based on code semantic representation and graph neural network, Comput. Secur. 118 (2022) 102715.
[138] M. Abu Talib, A. Bou Nassif, M. Azzeh, Y. Alesh, Y. Afadar, Parameter-efficient fine-tuning of pre-trained code models for just-in-time defect prediction, Neural Comput. Appl. 36 (27) (2024) 16911–16940.
[139] J. Han, C. Huang, J. Liu, Bjcnet: A contrastive learning-based framework for software defect prediction, Comput. Secur. 145 (2024) 104024.
[140] S. Yuan, C. Liu, J. Shi, X. Liu, W. Pu, J. Yu, L. Yang, A static detection method for code defects based on transformer, in: Proceedings of the 2024 3rd International Conference on Networks, Communications and Information Technology, 2024, pp. 104–111.
[141] F. Zhang, B. Chen, Y. Zhao, X. Peng, Slice-based code change representation learning, in: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2023, pp. 319–330.
[142] K.W. Nafi, B. Roy, C.K. Roy, K.A. Schneider, CroLSim: Cross language software similarity detector using api documentation, in: 2018 IEEE 18th International Working Conference on Source Code Analysis and Manipulation, SCAM, IEEE, 2018, pp. 139–148.
[143] Z. Xian, R. Huang, D. Towey, C. Fang, Z. Chen, TransformCode: a contrastive learning framework for code embedding via subtree transformation, IEEE Trans. Softw. Eng. (2024).

[144] S. Zhang, Y. Ding, E. Hu, Y. Yu, Y. Zhang, Enhancing code representation learning for code search with abstract code semantics, in: 2024 International Joint Conference on Neural Networks, IJCNN, IEEE, 2024, pp. 1–8.

[145] A. Grishina, M. Hort, L. Moonen, The earlybird catches the bug: On exploiting early layers of encoder models for more efficient code classification, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023, pp. 895–907.

[146] K. Pei, W. Li, Q. Jin, S. Liu, S. Geng, L. Cavallaro, J. Yang, S. Jana, Exploiting code symmetries for learning program semantics, in: Proceedings of the 41st International Conference on Machine Learning, ICML '24, JMLR.org, 2024, pp. 40092–40113.

[147] Z. Dong, Q. Hu, Z. Zhang, J. Zhao, On the effectiveness of graph data augmentation for source code learning, Knowl.-Based Syst. 285 (2024) 111328.

[148] P. Feng, D. Wei, Q. Li, Q. Wang, Y. Hu, N. Xi, J. Ma, GlareShell: Graph learning-based PHP webshell detection for web server of industrial internet, Comput. Netw. 245 (2024) 110406.

[149] A. Sharma, Z. Hu, C. Quinn, A. Jannesari, Redundancy and concept analysis for code-trained language models, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops, 2024, pp. 24–34.

[150] S. Liu, B. Wu, X. Xie, G. Meng, Y. Liu, Contrabert: Enhancing code pre-trained models via contrastive learning, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE, 2023, pp. 2476–2487.

[151] E. Dinella, H. Dai, Z. Li, M. Naik, L. Song, K. Wang, Hoppity: Learning graph transformations to detect and fix bugs in programs, in: International Conference on Learning Representations, ICLR, 2020, pp. 1–14.

[152] S.K. Pandey, M. Staron, J. Horkoff, M. Ochodek, N. Mucci, D. Durisic, Transdpr: Design pattern recognition using programming language models, in: 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM, IEEE, 2023, pp. 1–7.

[153] N.D. Bui, Y. Yu, L. Jiang, Bilateral dependency neural networks for cross-language algorithm classification, in: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2019, pp. 422–433.

[154] J.K. Siow, C. Gao, L. Fan, S. Chen, Y. Liu, Core: Automating review recommendation for code changes, in: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2020, pp. 284–295.

[155] J. Flisar, V. Podgorelec, Enhanced feature selection using word embeddings for self-admitted technical debt identification, in: 2018 44th Euromicro Conference on Software Engineering and Advanced Applications, SEAA, IEEE, 2018, pp. 230–233.

[156] C. Velázquez-Rodríguez, D. Di Nucci, C. De Roover, A text classification approach to API type resolution for incomplete code snippets, Sci. Comput. Program. 227 (2023) 102941.

[157] F. Zhang, B. Chen, R. Li, X. Peng, A hybrid code representation learning approach for predicting method names, J. Syst. Softw. 180 (2021) 111011.

[158] Y. Cao, Y. Dong, J. Peng, Vulnerability detection based on transformer and high-quality number embedding, Concurr. Comput.: Pr. Exp. 36 (28) (2024) e8292.

[159] S. Nguyen, T.T. Vu, H.D. Vo, VFFINDER: A graph-based approach for automated silent vulnerability-fix identification, in: 2023 15th International Conference on Knowledge and Systems Engineering, KSE, IEEE, 2023, pp. 1–6.

[160] R. Cabrera Lozoya, A. Baumann, A. Sabetta, M. Bezzi, Commit2vec: Learning distributed representations of code changes, SN Comput. Sci. 2 (3) (2021) 150.

[161] R. Li, B. Chen, F. Zhang, C. Sun, X. Peng, Detecting runtime exceptions by deep code representation learning with attention-based graph neural networks, in: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2022, pp. 373–384.

[162] R. Folea, R. Iacob, E. Slusanschi, T. Rebedea, Complexity-based code embeddings, in: International Conference on Computational Collective Intelligence, Springer, 2023, pp. 256–269.

[163] Z. Xu, M. Zhou, X. Zhao, Y. Chen, X. Cheng, H. Zhang, xASTNN: Improved code representations for industrial practice, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023, pp. 1727–1738.

[164] Y. Jiang, X. Su, C. Treude, T. Wang, Hierarchical semantic-aware neural code representation, J. Syst. Softw. 191 (2022) 111355.

[165] Y. Li, H. Wu, H. Zhao, Semantic-preserving adversarial code comprehension, in: N. Calzolari, C.-R. Huang, H. Kim, J. Pustejovsky, L. Wanner, K.-S. Choi, P.-M. Ryu, H.-H. Chen, L. Donatelli, H. Ji, S. Kurohashi, P. Paggio, N. Xue, S. Kim, Y. Hahm, Z. He, T.K. Lee, E. Santus, F. Bond, S.-H. Na (Eds.), Proceedings of the 29th International Conference on Computational Linguistics, International Committee on Computational Linguistics, Gyeongju, Republic of Korea, 2022, pp. 3017–3028, URL https://aclanthology.org/2022.coling-1.267/.

[166] W. Wang, K. Zhang, G. Li, S. Liu, A. Li, Z. Jin, Y. Liu, Learning program representations with a tree-structured transformer, in: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2023, pp. 248–259.

[167] Z. Liu, Z. Tang, X. Xia, X. Yang, Ccrep: Learning code change representations via pre-trained code model and query back, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE, 2023, pp. 17–29.

[168] H. Li, S. Wang, W. Quan, X. Gong, H. Su, J. Zhang, Prism: Decomposing program semantics for code clone detection through compilation, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13.

[169] D. Yuan, S. Fang, T. Zhang, Z. Xu, X. Luo, Java code clone detection by exploiting semantic and syntax information from intermediate code-based graph, IEEE Trans. Reliab. 72 (2) (2022) 511–526.

[170] B. Li, C. Ye, S. Guan, H. Zhou, Semantic code clone detection via event embedding tree and gat network, in: 2020 IEEE 20th International Conference on Software Quality, Reliability and Security, QRS, IEEE, 2020, pp. 382–393.

[171] A. Gupta, R. Goyal, A generative ai-driven method-level semantic clone detection based on the structural and semantical comparison of methods, IEEE Access (2024).

[172] S. Dou, Y. Wu, H. Jia, Y. Zhou, Y. Liu, Y. Liu, CC2vec: Combining typed tokens with contrastive learning for effective code clone detection, Proc. the ACM Softw. Eng. 1 (FSE) (2024) 1564–1584.

[173] C. Huang, H. Zhou, C. Ye, B. Li, Code clone detection based on event embedding and event dependency, in: Proceedings of the 13th Asia-Pacific Symposium on Internetware, 2022, pp. 65–74.

[174] Z. Li, P. Ma, H. Wang, S. Wang, Q. Tang, S. Nie, S. Wu, Unleashing the power of compiler intermediate representation to enhance neural program embeddings, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2253–2265.

[175] Y. Li, H. Wu, H. Zhao, Contrastive learning of functionality-aware code embeddings, in: ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, IEEE, 2023, pp. 1–5.

[176] Z. Lin, G. Li, J. Zhang, Y. Deng, X. Zeng, Y. Zhang, Y. Wan, Xcode: Towards cross-language code representation with large-scale pre-training, ACM Trans. Softw. Eng. Methodol. (TOSEM) 31 (3) (2022) 1–44.

[177] X. Zhang, S. Yang, L. Duan, Z. Lang, Z. Shi, L. Sun, Transformer-XL with graph neural network for source code summarization, in: 2021 IEEE International Conference on Systems, Man, and Cybernetics, SMC, IEEE, 2021, pp. 3436–3441.

[178] Z. Yu, W. Zhang, T. Xu, Leveraging IR based sequence and graph features for source-binary code alignment, in: 2024 4th International Conference on Neural Networks, Information and Communication, NNICE, IEEE, 2024, pp. 175–180.

[179] Q. Jia, M. Zhang, X. Guo, M. Liu, X. Tian, S. Peng, X. Jin, D. Ma, Research on open-source component homology analysis model based on graph neural network and attention mechanism, in: 2024 4th International Conference on Consumer Electronics and Computer Engineering, ICCECE, IEEE, 2024, pp. 225–229.

[180] H. Cheers, Y. Lin, S.P. Smith, A novel approach for detecting logic similarity in plagiarised source code, in: 2019 IEEE 10th International Conference on Software Engineering and Service Science, ICSESS, IEEE, 2019, pp. 1–6.

[181] H. Zhang, L. Zhang, L. Fang, R. Filgueira, Multi-level AI-driven analysis of software repository similarities, in: 2024 IEEE 20th International Conference on E-Science (E-Science), IEEE, 2024, pp. 1–10.

[182] H. Tian, K. Liu, Y. Li, A.K. Kaboré, A. Koyuncu, A. Habib, L. Li, J. Wen, J. Klein, T.F. Bissyandé, The best of both worlds: Combining learned embeddings with engineered features for accurate prediction of correct patches, ACM Trans. Softw. Eng. Methodol. 32 (4) (2023) 1–34.

[183] H. Tian, K. Liu, A.K. Kaboré, A. Koyuncu, L. Li, J. Klein, T.F. Bissyandé, Evaluating representation learning of code changes for predicting patch correctness in program repair, in: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, 2020, pp. 981–992.

[184] K.W. Nafi, B. Roy, C.K. Roy, K.A. Schneider, A universal cross language software similarity detector for open source software categorization, J. Syst. Softw. 162 (2020) 110491.

[185] K. Cheng, H. Wang, G. Hu, L. Zhang, J. Chen, W. Luo, Research on webshell detection based on semantic analysis and text-cnn, in: 2021 17th International Conference on Computational Intelligence and Security, CIS, IEEE, 2021, pp. 529–534.

[186] J. Zhang, X. Wang, N. Zheng, K. Huang, Y. Feng, X. Cui, Xshellgnn: Cross-file web shell detection based on graph neural network, in: 2024 27th International Conference on Computer Supported Cooperative Work in Design, CSCWD, IEEE, 2024, pp. 2607–2613.

[187] A.Y.M. Benselloua, S.A. Messadi, A.E. Belfedhal, Effective malicious PowerShell scripts detection using distilbert, in: 2023 IEEE Afro-Mediterranean Conference on Artificial Intelligence, AMCAI, IEEE, 2023, pp. 1–6.

[188] M. Geng, S. Wang, D. Dong, S. Gu, F. Peng, W. Ruan, X. Liao, Fine-grained code-comment semantic interaction analysis, in: Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension, 2022, pp. 585–596.

[189] R.S. Malik, J. Patra, M. Pradel, NL2type: Inferring JavaScript function types from natural language information, in: 2019 IEEE/ACM 41st International Conference on Software Engineering, ICSE, IEEE, 2019, pp. 304–315.

[190] X. Yi, G. Li, J. Li, A. Ding, Repfti: Representation-fused function-type inference for vehicular secure software systems, Appl. Sci. 14 (11) (2024) 4502.

[191] U. Alon, M. Zilberstein, O. Levy, E. Yahav, Code2vec: Learning distributed representations of code, Proc. the ACM Program. Lang. 3 (POPL) (2019) 1–29.

[192] Y. Shen, M. Peng, S. Wang, Q. Wu, Towards parallelism detection of sequential programs with graph neural network, Future Gener. Comput. Syst. 125 (2021) 515–525.

[193] Y. Hu, H. Jiang, Z. Hu, Measuring code maintainability with deep neural networks, Front. Comput. Sci. 17 (6) (2023) 176214.

[194] N. Bibi, A. Maqbool, T. Rana, F. Afzal, A. Akg¨ul, S.M. Eldin, Enhancing semantic code search with deep graph matching, IEEE Access 11 (2023) 52392–52411.

[195] Z. Deng, L. Xu, C. Liu, M. Yan, Z. Xu, Y. Lei, Fine-grained co-attentive representation learning for semantic code search, in: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2022, pp. 396–407.

[196] V. Efstathiou, D. Spinellis, Semantic source code models using identifier embeddings, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories, MSR, IEEE, 2019, pp. 29–33.

[197] Y. Zhang, Y. Liu, X. Fan, Y. Lu, CPLCS: Contrastive prompt learning-based code search with cross-modal interaction mechanism, in: 2024 International Joint Conference on Neural Networks, IJCNN, IEEE, 2024, pp. 1–10.

[198] X. Gu, H. Zhang, S. Kim, Deep code search, in: Proceedings of the 40th International Conference on Software Engineering, 2018, pp. 933–944.

[199] S. Sachdev, H. Li, S. Luan, S. Kim, K. Sen, S. Chandra, Retrieval on source code: a neural code search, in: Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, 2018, pp. 31–41.

[200] C. Zeng, Y. Yu, S. Li, X. Xia, Z. Wang, M. Geng, L. Bai, W. Dong, X. Liao, Degraphcs: Embedding variable-based flow graph for neural code search, ACM Trans. Softw. Eng. Methodol. 32 (2) (2023) 1–27.

[201] W. Gu, Z. Li, C. Gao, C. Wang, H. Zhang, Z. Xu, M.R. Lyu, Cradle: Deep code retrieval based on semantic dependency learning, Neural Netw. 141 (2021) 385–394.

[202] P. Salza, C. Schwizer, J. Gu, H.C. Gall, On the effectiveness of transfer learning for code search, IEEE Trans. Softw. Eng. 49 (4) (2022) 1804–1822.

[203] W. Zhao, Y. Liu, Utilizing edge attention in graph-based code search., in: SEKE, 2022, pp. 60–66.

[204] N. Bibi, A. Maqbool, T. Rana, Exploring the effectiveness of joint bi-LSTM-GNN vs ChatGPT-LLM in source code retrieval, in: 2023 20th International Bhurban Conference on Applied Sciences and Technology, IBCAST, IEEE, 2023, pp. 321–326.

[205] J. Hong, E. Choi, O. Mizuno, GNEM: Comprehensive similarity learning with ensemble model for code search, IEEE Access (2024).

[206] S. Zhang, L. Shuiyan, Q. Rongzhi, X. Zhou, CodeFuse: Multimodal code search model with fine-grained attention alignment, in: 2024 IEEE 48th Annual Computers, Software, and Applications Conference, COMPSAC, IEEE, 2024, pp. 1290–1299.

[207] J. Hong, E. Choi, K. Nishiura, O. Mizuno, Global alignment learning for code search, in: 2024 IEEE/ACIS 22nd International Conference on Software Engineering Research, Management and Applications, SERA, IEEE, 2024, pp. 334–339.

[208] S. Liu, W. Guo, Y. Yu, B. Cai, Enhancing code search with token-level information flow graphs generated from aligned structural and textual features, in: 2023 IEEE International Conference on Systems, Man, and Cybernetics, SMC, IEEE, 2023, pp. 2283–2288.

[209] C. Chen, Z. Xing, Y. Liu, K.O.L. Xiong, Mining likely analogical apis across third-party libraries via large-scale unsupervised api semantics embedding, IEEE Trans. Softw. Eng. 47 (3) (2019) 432–447.

[210] N.D. Bui, Y. Yu, L. Jiang, Self-supervised contrastive learning for code retrieval and summarization via semantic-preserving transformations, in: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2021, pp. 511–521.

[211] J. Zhang, R. Xie, W. Ye, Y. Zhang, S. Zhang, Exploiting code knowledge graph for bug localization via bi-directional attention, in: Proceedings of the 28th International Conference on Program Comprehension, 2020, pp. 219–229.

[212] J. Yong, Z. Zhu, Y. Li, Decomposing source codes by program slicing for bug localization, in: 2023 International Joint Conference on Neural Networks, IJCNN, IEEE, 2023, pp. 1–8.

[213] M. Farzandway, F. Ghassemi, Specnlp: A pre-trained model enhanced with spectrum profile for bug localization, in: 2024 IEEE International Conference on Artificial Intelligence Testing (AITest), IEEE, 2024, pp. 81–86.

[214] Y. Xiao, J. Keung, K.E. Bennin, Q. Mi, Improving bug localization with word embedding and enhanced convolutional neural networks, Inf. Softw. Technol. 105 (2019) 17–29.

[215] W. Ali, L. Bo, X. Sun, X. Wu, S. Memon, S. Siraj, A.S. Ashton, Automated software bug localization enabled by meta-heuristic-based convolutional neural network and improved deep neural network, Expert Syst. Appl. 232 (2023) 120562.

[216] Z. Zhu, H. Tong, Y. Wang, Y. Li, Enhancing bug localization with bug report decomposition and code hierarchical network, Knowl.-Based Syst. 248 (2022) 108741.

[217] B. Wu, F. Zou, P. Yi, Y. Wu, L. Zhang, SlicedLocator: Code vulnerability locator based on sliced dependence graph, Comput. Secur. 134 (2023) 103469.

[218] T.-T. Nguyen, H.D. Vo, Context-based statement-level vulnerability localization, Inf. Softw. Technol. 169 (2024) 107406.

[219] J. Han, C. Huang, S. Sun, Z. Liu, J. Liu, Bjxnet: an improved bug localization model based on code property graph and attention mechanism, Autom. Softw. Eng. 30 (1) (2023) 12.

[220] H. Liang, L. Sun, M. Wang, Y. Yang, Deep learning with customized abstract syntax tree for bug localization, IEEE Access 7 (2019) 116309–116320.

[221] R. Compton, E. Frank, P. Patros, A. Koay, Embedding java classes with code2vec: Improvements from variable obfuscation, in: Proceedings of the 17th International Conference on Mining Software Repositories, 2020, pp. 243–253.

[222] C. Wang, X. Peng, Z. Xing, X. Meng, Beyond literal meaning: Uncover and explain implicit knowledge in code through wikipedia-based concept linking, IEEE Trans. Softw. Eng. 49 (5) (2023) 3226–3240.

[223] N.D. Bui, Y. Yu, L. Jiang, Infercode: Self-supervised learning of code representations by predicting subtrees, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering, ICSE, IEEE, 2021, pp. 1186–1197.

[224] Y. Sui, X. Cheng, G. Zhang, H. Wang, Flow2vec: Value-flow-based precise code embedding, Proc. the ACM Program. Lang. 4 (OOPSLA) (2020) 1–27.

[225] S. VenkataKeerthy, R. Aggarwal, S. Jain, M.S. Desarkar, R. Upadrasta, Y. Srikant, Ir2vec: Llvm ir based scalable program embeddings, ACM Trans. Archit. Code Optim. (TACO) 17 (4) (2020) 1–27.

[226] A. Kicsi, V. Csuvik, L. Vidács, Large scale evaluation of natural language processing based test-to-code traceability approaches, IEEE Access 9 (2021) 79089–79104.

[227] N. Nashid, M. Sintaha, A. Mesbah, Embedding context as code dependencies for neural program repair, in: 2023 IEEE Conference on Software Testing, Verification and Validation, ICST, IEEE, 2023, pp. 95–106.

[228] J. Patra, M. Pradel, Semantic bug seeding: a learning-based approach for creating realistic bugs, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 906–918.

[229] Y. Xiao, Multi-location cryptographic code repair with neural-network-based methodologies, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 1640–1644.

[230] J. Kim, J. Jeon, S. Hong, S. Yoo, Predictive mutation analysis via the natural language channel in source code, ACM Trans. Softw. Eng. Methodol. (TOSEM) 31 (4) (2022) 1–27.

[231] J. Zhang, S. Liu, L. Gong, H. Zhang, Z. Huang, H. Jiang, Beqain: An effective and efficient identifier normalization approach with bert and the question answering system, IEEE Trans. Softw. Eng. 49 (4) (2022) 2597–2620.

[232] L. Dramko, J. Lacomis, P. Yin, E. Schwartz, M. Allamanis, G. Neubig, B. Vasilescu, C. Le Goues, DIRE and its data: Neural decompiled variable renamings with respect to software class, ACM Trans. Softw. Eng. Methodol. 32 (2) (2023) 1–34.

[233] N.D. Bui, L. Jiang, Hierarchical learning of cross-language mappings through distributed vector representations for code, in: Proceedings of the 40th International Conference on Software Engineering: New Ideas and Emerging Results, 2018, pp. 33–36.

[234] D. Cui, S. Wang, Y. Luo, X. Li, J. Dai, L. Wang, Q. Li, Rmove: Recommending move method refactoring opportunities using structural and semantic representations of code, in: 2022 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2022, pp. 281–292.

[235] H. Thaller, L. Linsbauer, A. Egyed, Feature maps: A comprehensible software representation for design pattern detection, in: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2019, pp. 207–217.

[236] H. Chen, Y. Huang, Z. Liu, X. Chen, F. Zhou, X. Luo, Automatically detecting the scopes of source code comments, J. Syst. Softw. 153 (2019) 45–63.

[237] S. Phatangare, A. Matkar, A. Jadhav, A.H. Shaikh, A. Bonde, CodeCompass: NLP-driven navigation to optimal repositories, in: 2024 4th International Conference on Pervasive Computing and Social Networking, ICPCSN, IEEE, 2024, pp. 393–401.

[238] Z. Huang, J. Chen, J. Jiang, Y. Liang, H. You, F. Li, Mapping APIs in dynamic-typed programs by leveraging transfer learning, ACM Trans. Softw. Eng. Methodol. 33 (4) (2024) 1–29.

[239] W. Ma, S. Liu, M. Zhao, X. Xie, W. Wang, Q. Hu, J. Zhang, Y. Liu, Unveiling code pre-trained models: Investigating syntax and semantics capacities, ACM Trans. Softw. Eng. Methodol. 33 (7) (2024) 1–29.

[240] J. Song, Z. Zhang, Z. Tang, S. Feng, Y. Gu, Improving code summarization with tree transformer enhanced by position-related syntax complement, IEEE Trans. Artif. Intell. 5 (9) (2024) 4776–4786.

[241] J.A. Li, Y. Li, G. Li, X. Hu, X. Xia, Z. Jin, Editsum: A retrieve-and-edit framework for source code summarization, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2021, pp. 155–166.

[242] X. Gao, X. Jiang, Q. Wu, X. Wang, C. Lyu, L. Lyu, GT-SimNet: Improving code automatic summarization via multi-modal similarity networks, J. Syst. Softw. 194 (2022) 111495.

[243] C. Fang, W. Sun, Y. Chen, X. Chen, Z. Wei, Q. Zhang, Y. You, B. Luo, Y. Liu, Z. Chen, Esale: Enhancing code-summary alignment learning for source code summarization, IEEE Trans. Softw. Eng. (2024).
[244] K. Vikram, J.V. Balaji, V. Vani, et al., CodeSense: Code summarizer for JavaScript, in: 2024 International Conference on Signal Processing, Computation, Electronics, Power and Telecommunication (IConSCEPT), IEEE, 2024, pp. 1–5.
[245] W. Sun, Y. Hu, Y. Xu, Y. Chen, C. Fang, Integrating extractive and abstractive models for code comment generation, in: 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security, QRS, IEEE, 2023, pp. 184–195.
[246] R. Wang, H. Zhang, G. Lu, L. Lyu, C. Lyu, Fret: Functional reinforced transformer with bert for code summarization, IEEE Access 8 (2020) 135591–135604.
[247] X. Li, S. Liu, R. Feng, G. Meng, X. Xie, K. Chen, Y. Liu, Transrepair: Context-aware program repair for compilation errors, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–13.
[248] Y. Li, S. Wang, T.N. Nguyen, Dear: A novel deep learning-based approach for automated program repair, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 511–523.
[249] F. Mu, X. Chen, L. Shi, S. Wang, Q. Wang, Developer-intent driven code comment generation, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE, 2023, pp. 768–780.
[250] Y. Zhou, X. Zhang, J. Shen, T. Han, T. Chen, H. Gall, Adversarial robustness of deep code comment generation, ACM Trans. Softw. Eng. Methodol. (TOSEM) 31 (4) (2022) 1–30.
[251] X. Hu, G. Li, X. Xia, D. Lo, Z. Jin, Deep code comment generation with hybrid lexical and syntactical information, Empir. Softw. Eng. 25 (2020) 2179–2217.
[252] C. Wang, Y. Wu, X. Zhang, Mucha: Multi-channel based code change representation learning for commit message generation, in: 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security, QRS, IEEE, 2023, pp. 593–603.
[253] Z. Li, C. Zhang, M. Pan, T. Zhang, X. Li, Aacegen: Attention guided adversarial code example generation for deep code models, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1245–1257.
[254] Y. Gu, Y. Shen, X. Chen, S. Yang, Y. Huang, Z. Cao, Apicom: Automatic api completion via prompt learning and adversarial training-based data augmentation, in: Proceedings of the 14th Asia-Pacific Symposium on Internetware, 2023, pp. 259–269.
[255] X. Li, L. Liu, Y. Liu, H. Liu, A lightweight API recommendation method for app development based on multi-objective evolutionary algorithm, Sci. Comput. Program. 226 (2023) 102927.